

SKALA

KDT Backend

Java 21 핵심

2026.07

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Java 와 Java 21

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Java 언어의 특징

- 1995년 등장 이후 30년 넘게 서버 애플리케이션의 주력 언어로 쓰이고 있다
- 핵심 약속은 WORA — 소스가 아니라 바이트코드를 배포하고, OS 별 JVM 이 그것을 실행한다

특징	무엇을 뜻하는가
객체지향	데이터(상태)와 동작(기능)을 객체로 묶어 설계한다
플랫폼 독립	소스를 바이트코드로 컴파일하고, OS 별 JVM 이 그것을 실행한다
자동 메모리 관리	가비지 컬렉터(GC)가 더 이상 쓰지 않는 객체를 회수한다
강한 타입 검사	타입이 맞지 않는 코드 상당수를 실행 전 컴파일 시점에 잡아낸다
풍부한 표준 라이브러리	컬렉션 · 동시성 · 네트워크 · 날짜/시간이 기본으로 들어 있다
거대한 생태계	Spring · Gradle · JUnit 등 검증된 도구가 서버 개발 표준으로 자리잡았다
하위 호환성	오래전에 만든 코드가 새 JVM 에서도 대체로 그대로 동작한다

자바는 느리다? 옛말이다 — JIT 컴파일러 덕분

처음엔 바이트코드를 한 줄씩 해석해 조금 느리지만, 자주 실행되는 코드는 JIT(Just-In-Time) 컴파일러가 '실행 도중' 기계어로 바꿔 저장한다.

그래서 오래 켜 두는 서버에서는 초반의 느림이 금방 사라져 C 계열과 큰 차이가 없다. (짧게 한 번 돌리는 프로그램은 예외 전이라 예외)

초보 팁: 속도보다 '정확히 동작하는 코드'가 먼저다. 성능은 추측하지 말고 측정한 뒤 고친다.

Java 버전 흐름과 LTS

- 6개월마다 새 버전이 나오고, 그중 일부만 LTS(장기 지원) 버전으로 지정된다
- 운영 시스템은 보안 패치 기간이 긴 LTS 를 고른다 — 8 · 11 · 17 · 21 · 25

버전	출시	지원	이 버전에서 들어온 대표 기능
Java 8	2014.03	LTS	람다식 · 스트림 · Optional · java.time — ‘모던 자바’의 출발점
Java 11	2018.09	LTS	지역 변수 var · HTTP Client · 단일 파일 소스 실행
Java 17	2021.09	LTS	record · sealed · switch 표현식 · 텍스트 블록
Java 21	2023.09	LTS	가상 스레드 · Sequenced Collections · switch 패턴 매칭 · record 패턴
Java 25	2025.09	LTS	그다음 장기 지원 버전

왜 LTS(장기 지원) 버전을 고르나

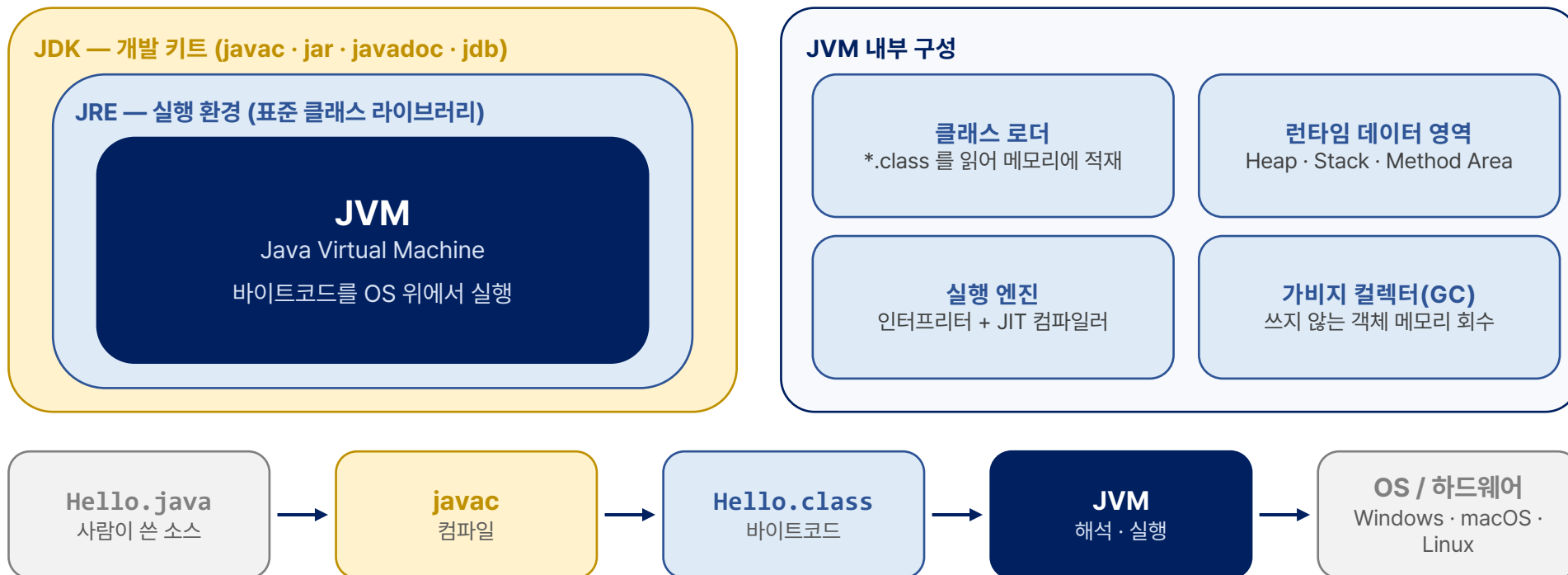
자바는 6개월마다 새 버전이 나오지만, 일반 버전은 보안 패치 지원이 6개월이면 끝난다.

LTS 버전(8·11·17·21...)은 수년간 패치를 받아 안정적이라, 실무 운영 시스템은 대부분 LTS 를 쓴다.

이 교안은 Java 21 기준. 21 에는 8·11·17 의 기능이 모두 들어 있어, 21 만 익히면 그동안의 개선을 한꺼번에 얻는다.

[도식] Java 실행 구조 — JDK · JRE · JVM

- 자바 소스는 OS 가 아니라 바이트코드로 컴파일되고, OS 별 JVM 이 그 차이를 흡수한다



Write Once, Run Anywhere — 한 번 만들면 어디서나 실행

자바 소스(.java)는 특정 OS 가 아니라 '바이트코드(.class)'로 컴파일된다. OS 마다 있는 JVM 이 그 바이트코드를 각자의 기계어로 실행해 준다.

포함 관계 JDK ⊃ JRE ⊃ JVM : 개발하려면 JDK(컴파일러 javac 포함), 실행만 하려면 JRE, 실제로 코드를 돌리는 심장은 JVM 이다.

Spring Boot 의 실행 JAR 도 결국 JVM 위에서 도는 바이트코드 묶음이다 → java -jar app.jar

개발 환경 구성

- JDK · 편집기 · 빌드 도구 세 가지면 시작할 수 있다

구성 요소	무엇을 설치하나	설치 후 확인
JDK 21	Eclipse Temurin 21 또는 Amazon Corretto 21 (둘 다 무료)	<code>java -version</code>
편집기	VS Code + Extension Pack for Java	Java 파일에서 자동완성 동작
빌드 도구	Gradle (Spring Boot 기본) 또는 Maven	<code>gradle -v</code>
컴파일러	JDK 에 포함되어 별도 설치가 필요 없다	<code>javac -version</code>

설치 확인

```
$ java -version
openjdk version "21.0.x" ...    ← 첫 줄이 21 로 시작하면 정상

$ javac -version
javac 21.0.x
```

JAVA_HOME 환경 변수를 먼저 잡아 두자

JAVA_HOME 은 'JDK 가 설치된 위치'를 알려 주는 환경 변수다. Gradle·IDE·터미널 스크립트가 이 값으로 JDK 를 찾는다.

확인: 터미널에서 `java -version` 을 쳐서 21.x 가 보이면 성공.

여러 자바 버전을 오갈 땐 SDKMAN!(macOS/Linux)·jEnv 같은 버전 관리 도구가 편하다.

첫 프로그램 — 작성부터 실행까지

- 클래스 안에 main 메서드를 두면 JVM 이 그곳부터 실행한다

Hello.java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java 21!");
    }
}
```

컴파일하고 실행하기

```
$ javac Hello.java      # 컴파일 → Hello.class 생성
$ java Hello            # 실행 (.class 확장자는 빼고)
Hello, Java 21!

$ java Hello.java        # Java 11+ : 한 번에 실행
Hello, Java 21!
```

main 메서드의 각 조각	왜 그렇게 써야 하나
public	JVM 이 클래스 밖에서 호출해야 하므로 공개되어 있어야 한다
static	객체를 만들기 전에 호출되므로 인스턴스가 필요 없어야 한다
void	JVM 에 돌려줄 값이 없다 (종료 코드는 System.exit 로 준다)
String[] args	실행할 때 넘긴 명령행 인자를 배열로 받는다

[핵심 요약] Java 와 Java 21

Java 를 쓰는 이유

플랫폼 독립 · 자동 메모리 관리 · 강한 타입 검사
그리고 서버 개발에 검증된 거대한 생태계

LTS 를 고르는 이유

일반 버전은 지원이 6개월뿐이다
8 → 11 → 17 → 21 → 25 가 장기 지원 버전

실행 구조

소스(.java) → 바이트코드(.class) → JVM 이 실행
JDK ⊃ JRE ⊃ JVM · 개발하려면 JDK 가 필요하다

최소 개발 환경

JDK 21 · 편집기 · 빌드 도구(Gradle)
java -version 으로 21 인지 먼저 확인한다

Java 21 = 그동안의 문법 개선을 한꺼번에

Java 21 을 쓴다는 것은 21 의 신기능뿐 아니라 9~21 사이에 쌓인 모든 문법 개선(var-switch 표현식·record·텍스트 블록 등)을 함께 쓴다는 뜻이다.

즉 '최신 버전 하나'가 아니라 '10여 년치 편의 기능 묶음'을 얻는 것이다.

다음 PART 부터 이 기능들을 예제와 함께 하나씩 익힌다.

언어 기초

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

변수와 기본 데이터 타입

- 자바의 타입은 값을 직접 담는 기본형 8종과, 객체의 주소를 담는 참조형으로 나뉜다

분류	타입	크기	표현 범위 · 특징	예
정수	byte	1 byte	-128 ~ 127	byte b = 100;
	short	2 byte	-32,768 ~ 32,767	short s = 1000;
	int	4 byte	약 ±21억 — 정수의 기본 타입	int i = 100_000;
	long	8 byte	매우 큰 정수 — 리터럴에 L 을 붙인다	long l = 100L;
실수	float	4 byte	오차가 있는 근삿값 — F 를 붙인다	float f = 1.5F;
	double	8 byte	실수의 기본 타입 — 역시 근삿값	double d = 1.5;
문자	char	2 byte	유니코드 문자 하나 — 작은따옴표	char c = 'A';
논리	boolean	1 bit	true / false	boolean ok = true;

돈 계산에 double 을 쓰지 않는다

0.1 + 0.2 == 0.3 이 false 다. 부동소수점은 2진수 근삿값이기 때문. 금액·수량은 BigDecimal 을 쓰거나 정수(원 단위)로 다룬다.

기본형 vs 참조형

- 대입할 때 기본형은 값이 복사되고, 참조형은 주소가 복사된다
- 이 차이를 모르면 '분명 안 건드렸는데 값이 바뀌는' 버그를 만난다

기본형 (primitive) – 값 자체를 담는다

```
int a = 10;
int b = a;    // 값이 복사된다
b = 20;
// a 는 여전히 10
```

- null 이 될 수 없다
- 스택에 값이 그대로 놓인다

참조형 (reference) – 객체의 주소를 담는다

```
int[] a = {1, 2, 3};
int[] b = a;    // 주소가 복사된다
b[0] = 99;
// a[0] 도 99 가 된다 (같은 객체)
```

- null 이 될 수 있다
- 실제 객체는 힙에 있다

기본형	래퍼 클래스	박싱 / 언박싱
int	Integer	컬렉션·제네릭에는 객체만 담기므로 자동으로 Integer 로 감싼다(오토박싱)
long / double	Long / Double	반복문 안에서 잦은 박싱은 성능을 떨어뜨린다 — IntStream 등을 쓴다
boolean / char	Boolean / Character	래퍼는 null 이 될 수 있어, 언박싱 시 NullPointerException 이 난다

형 변환과 var

- 작은 타입 → 큰 타입은 자동, 반대는 명시적 캐스팅이 필요하고 값이 잘릴 수 있다

형 변환 (casting)

```
int i = 100;
long l = i;           // 자동 (작은 → 큰)

double d = 3.9;
int n = (int) d;       // 명시적, n == 3 (버림)

// 문자열 ↔ 숫자
int x = Integer.parseInt("42");
String s = String.valueOf(42);
```

var – 지역 변수 타입 추론 (Java 10+)

```
var name = "홍길동";           // String
var count = 10;                 // int
var users = new ArrayList<User>();

// 쓸 수 없는 자리
// var x;                       선언만 하면 추론 불가
// var f = null;                 타입을 알 수 없다
// 필드 · 메서드 파라미터 · 반환 타입에는 불가
```

var 는 '타입이 사라지는 것'이 아니라 '타입을 컴파일러가 대신 적어 주는 것'이다

오른쪽만 봐도 타입이 분명할 때(new ArrayList<User>()) 쓰면 읽기 좋아지고, 오른쪽이 메서드 호출뿐이라 타입이 안 보일 때는 오히려 타입을 적어 주는 편이 낫다.

연산자

- 종류보다 중요한 것은 '정수 나눗셈'과 '참조 비교', 그리고 논리 연산의 단축 평가다

분류	연산자	알아 둘 점
산술	+ - * / %	정수끼리의 / 는 몫만 남는다 (5 / 2 == 2). 0 으로 나누면 ArithmeticException
비교	== != > < >= <=	참조형에 == 를 쓰면 '같은 객체인가'를 본다 — 값 비교는 equals()
논리	&& !	&& 는 앞이 false 면 뒤를 아예 평가하지 않는다(단축 평가) — null 검사에 활용
대입	= += -= *= /=	a += 1 은 a = a + 1 의 축약
증감	++ --	i++ 는 쓰고 나서 증가, ++i 는 증가하고 나서 쓴다
삼항	조건 ? A : B	간단한 분기에만 — 중첩하면 급격히 읽기 어려워진다
기타	instanceof -> ::	타입 검사 · 람다 · 메서드 참조. 뒤에서 자세히 다룬다

NULL 검사와 단축 평가

if (s != null && s.length() > 0) — 앞이 false 면 뒤를 평가하지 않는다. 순서를 바꾸면 NPE.

문자열 — String 과 StringBuilder

- String 은 한 번 만들면 바꿀 수 없다(불변). 이어 붙일 때마다 새 객체가 생긴다

String 은 불변(immutable) 이다

```
String s = "Java";
s = s + " 21";    // 새 String 객체가 만들어진다

// 반복문에서 += 로 이어 붙이면
// 매번 새 객체가 생겨 느려진다

"a".equals(s)    // 값 비교는 equals
s == "a"         // 참조 비교 - 쓰지 말 것
```

많이 이어 붙일 때는 StringBuilder

```
var sb = new StringBuilder();
for (var u : users) {
    sb.append(u.name()).append(", ");
}
String result = sb.toString();

// 문자열 사이에 구분자를 넣을 땐
String.join(", ", names);
```

자주 쓰는 메서드	설명
isBlank() / strip()	공백만 있는지 검사 / 앞뒤 공백 제거 (Java 11+, trim 보다 유니코드에 정확)
formatted(...)	"%s님 안녕하세요".formatted(name) — String.format 의 인스턴스 버전
repeat(n) / lines()	문자열 반복 / 줄 단위 스트림으로 분해 (Java 11+)
contains · startsWith · split	포함 여부 · 접두사 검사 · 구분자로 자르기

텍스트 블록 (Java 15+)

- 여러 줄 문자열을 이스케이프 없이 보이는 그대로 쓸 수 있다

예전 방식 - 이스케이프 지옥

```
String json = "{\n"
    + "  \"name\": \"홍길동\",\n"
    + "  \"age\": 20\n"
    + "}";

String sql = "SELECT id, name "
    + "FROM users "
    + "WHERE age >= ?";
```

텍스트 블록 (Java 15+)

```
String json = """
    {
      "name": "홍길동",
      "age": 20
    }""";

String sql = """
    SELECT id, name
    FROM users
    WHERE age >= ?""";
```

들여쓰기는 '가장 왼쪽 줄'을 기준으로 자동으로 건너낸다

닫는 """" 의 위치도 기준에 포함된다. 줄 끝에 \ 를 붙이면 줄바꿈을 없앨 수 있고, \s 로 뒤쪽 공백을 지킬 수 있다.

JSON · SQL · HTML 처럼 여러 줄 문자열을 다룰 때 가독성이 크게 달라진다.

조건문 — if 와 switch 문

- 범위·복합 조건은 if, 값이 딱 떨어지는 분기는 switch 가 읽기 좋다

if – 조건이 범위·복합 조건일 때

```
if (score >= 90) {  
    grade = "A";  
} else if (score >= 80) {  
    grade = "B";  
} else {  
    grade = "C";  
}  
  
// 중괄호는 한 줄이어도 붙이는 편이 안전하다
```

switch 문 – 값이 딱 떨어질 때

```
switch (day) {  
    case MONDAY:  
    case TUESDAY:  
        System.out.println("주중");  
        break;    // 빠뜨리면 아래로 흘러내린다  
    default:  
        System.out.println("그 외");  
}
```

전통적인 switch 문의 세 가지 함정

- ① break 를 빠뜨리면 다음 case 로 흘러내린다(fall-through)
- ② case 안에서 선언한 변수의 범위가 switch 전체다
- ③ 값을 만들어 변수에 담으려면 밖에 변수를 미리 선언해야 한다 — 이 세 가지를 switch 표현식이 해결한다.

switch 표현식 (Java 14+)

- switch 가 문장이 아니라 '값을 만들어 내는 식'이 되면서 실수할 여지가 크게 줄었다

switch 표현식 (Java 14+) - 값을 돌려준다

```
String type = switch (day) {  
    case MONDAY, TUESDAY, WEDNESDAY,  
        THURSDAY, FRIDAY -> "주중";  
    case SATURDAY, SUNDAY -> "주말";  
};  
  
// -> 화살표는 fall-through 가 없다  
// break 를 쓸 필요가 없다  
// enum 을 모두 다루면 default 도 필요 없다
```

블록이 필요하면 yield

```
int len = switch (type) {  
    case "SHORT" -> 10;  
    case "LONG" -> {  
        var base = readBase();  
        yield base * 2;    // 블록의 결과값  
    }  
    default -> throw new IllegalArgumentException(type);  
};
```

switch 표현식은 '반드시 값을 만들어 내야' 하므로 컴파일러가 빠진 경우를 잡아 준다

값을 돌려주는 형태로 쓰면, 변수를 미리 선언했다가 분기마다 대입하는 패턴이 사라진다. Java 21 에서는 여기에 패턴 매칭이 더해진다.

반복문

- 인덱스가 꼭 필요한 경우가 아니라면 향상된 for(for-each)를 기본으로 쓴다

for · while

```
for (int i = 0; i < 5; i++) { ... }

int i = 0;
while (i < 5) { ... i++; }

do { ... } while (조건);    // 최소 1회 실행

// break : 반복 종료 / continue : 다음 회차로
```

향상된 for (for-each) – 기본으로 이것을 쓴다

```
for (String name : names) {
    System.out.println(name);
}

for (var entry : map.entrySet()) {
    entry.getKey(); entry.getValue();
}

// 인덱스가 정말 필요할 때만 일반 for 를 쓴다
```

반복 중에 컬렉션을 수정하면 ConcurrentModificationException 이 난다

for-each 를 돌면서 list.remove(...) 를 부르면 터진다. 조건에 맞는 항목을 지우려면 list.removeIf(조건) 을 쓰거나, 새 리스트를 만든다.

배열

- 배열은 크기가 고정된 연속 공간이다. 크기가 변하는 데이터에는 List 를 쓴다

배열 - 크기가 고정된 연속 공간

```
int[] nums = new int[5];        // 0 으로 초기화
int[] nums2 = {1, 2, 3, 4, 5};
String[][] grid = new String[3][4]; // 2차원

nums2.length        // 길이 (메서드가 아니라 필드)
nums2[0]             // 첫 번째
nums2[nums2.length - 1] // 마지막

// 범위를 벗어나면 ArrayIndexOutOfBoundsException
```

배열 유틸 · 컬렉션으로 넘어가기

```
Arrays.sort(nums2);
Arrays.toString(nums2);        // 출력용 문자열
Arrays.copyOf(nums2, 10);
Arrays.stream(nums2).sum();

List<String> list = Arrays.asList(arr); // 고정 크기
List<String> list2 = List.of("a", "b"); // 불변

// 크기가 변하는 데이터는 배열 대신 List 를 쓴다
```

실무에서 배열을 직접 쓰는 일은 생각보다 적다

크기를 미리 알 수 없는 데이터, 추가·삭제가 잦은 데이터는 List 가 훨씬 다루기 쉽다. 배열은 성능이 중요한 저수준 코드나 byte[] 같은 곳에서 주로 만난다.

[핵심 요약] 언어 기초

기본형과 참조형

기본형은 값을, 참조형은 주소를 담는다
참조형 비교는 `==` 가 아니라 `equals()`

var

지역 변수에만 쓸 수 있는 타입 추론
타입이 한눈에 보일 때만 쓴다

문자열

`String` 은 불변 — 많이 이어 붙이면 `StringBuilder`
여러 줄 문자열은 텍스트 블록 `""" """`

분기와 반복

값을 만들 땐 `switch` 표현식(`->`)
컬렉션 순회는 `for-each` 를 기본으로

정수 나눗셈은 몫만 남고, 실수는 근삿값이며, 문자열 비교는 `equals` — 초보 시절 버그의 절반은 이 세 가지에서 나온다

객체지향 프로그래밍

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

클래스와 객체

- 클래스는 설계도, 객체는 그 설계도로 만든 실체다
- 필드는 객체의 상태이고, 메서드는 객체의 행위다

클래스 - 설계도

```
public class Car {

    private String brand;    // 필드 = 상태
    private int speed;

    public void accelerate(int v) {    // 메서드 = 행위
        this.speed += v;
    }
    public int getSpeed() { return speed; }
}
```

객체 - 설계도로 만든 실체

```
Car car1 = new Car();
Car car2 = new Car();

car1.accelerate(60);

car1.getSpeed();    // 60
car2.getSpeed();    // 0 ← 서로 다른 상태

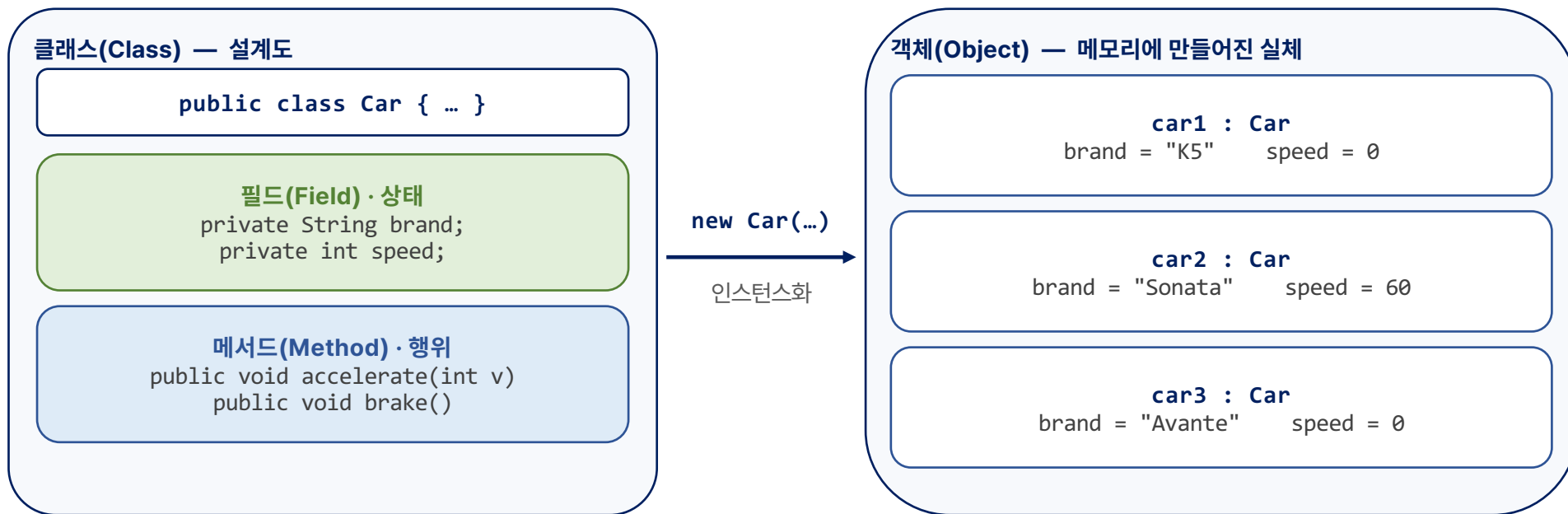
// 클래스 하나로 객체는 얼마든지 만들 수 있고,
// 각 객체는 자기만의 필드 값을 가진다
```

클래스를 나눌 때의 기준은 '무엇을 담는가'보다 '무엇을 책임지는가'다

필드를 몇 개 묶어 놓는 것이 목적이 아니라, 그 데이터를 다루는 규칙까지 한곳에 모으는 것이 객체지향의 출발점이다.

[도식] OOP 핵심 요소 관계도

- 하나의 클래스로 객체를 여러 개 만들 수 있고, 각 객체는 자신만의 필드 값을 가진다



OOP 핵심 용어 한 줄 정리

클래스(Class) : 객체를 만들기 위한 설계도 · 필드(Field) : 객체가 가지는 상태(데이터) · 메서드(Method) : 객체가 하는 행위(기능)

객체(Object) : 클래스로부터 만들어진 실체 · 인스턴스(Instance) : '특정 클래스로부터 생성되었다'는 관계를 강조한 표현

하나의 클래스로 객체를 여러 개 만들 수 있고, 각 객체는 자신만의 필드 값을 따로 가진다. 메서드(코드)는 공유된다.

생성자와 this

- 생성자는 객체가 만들어질 때 딱 한 번 실행되는, 초기화 전용 메서드다

생성자 - 객체가 만들어질 때 한 번 실행

```
public class User {
    private final String name;
    private final int age;

    public User(String name, int age) {
        if (age < 0) throw new IllegalArgumentException();
        this.name = name;    // this = 지금 만들어지는 객체
        this.age = age;
    }
}
```

오버로딩과 this(...) 위임

```
public User(String name) {
    this(name, 0);    // 다른 생성자 호출 (첫 줄에서만)
}

// 생성자를 하나도 안 쓰면 기본 생성자가 자동 생성
// 하나라도 직접 쓰면 자동 생성되지 않는다

// final 필드는 생성자에서 반드시 값이 정해져야 하고,
// 이후에는 바꿀 수 없다 → 불변 객체의 기초
```

생성자는 '올바른 객체만 태어나게 하는 문지기'다

여기서 값을 검증해 두면, 이후 코드 어디에서도 '나이가 음수인 User'를 걱정하지 않아도 된다. 검증을 setter 로 미루면 확인할 곳이 늘어난다.

접근 제어자와 캡슐화

- 데이터는 감추고(private), 검증된 통로(public 메서드)로만 접근하게 만든다

접근 제어자	같은 클래스	같은 패키지	자식 클래스	그 외	언제 쓰나
private	○	×	×	×	필드는 기본적으로 private
(default)	○	○	×	×	패키지 내부 구현용
protected	○	○	○	×	상속받은 쪽에서만 쓸 확장 지점
public	○	○	○	○	외부에 공개하는 API

캡슐화 : 필드는 private, 통로는 public 메서드

```
public class Account {
    private long balance;           // 상태는 감춘다

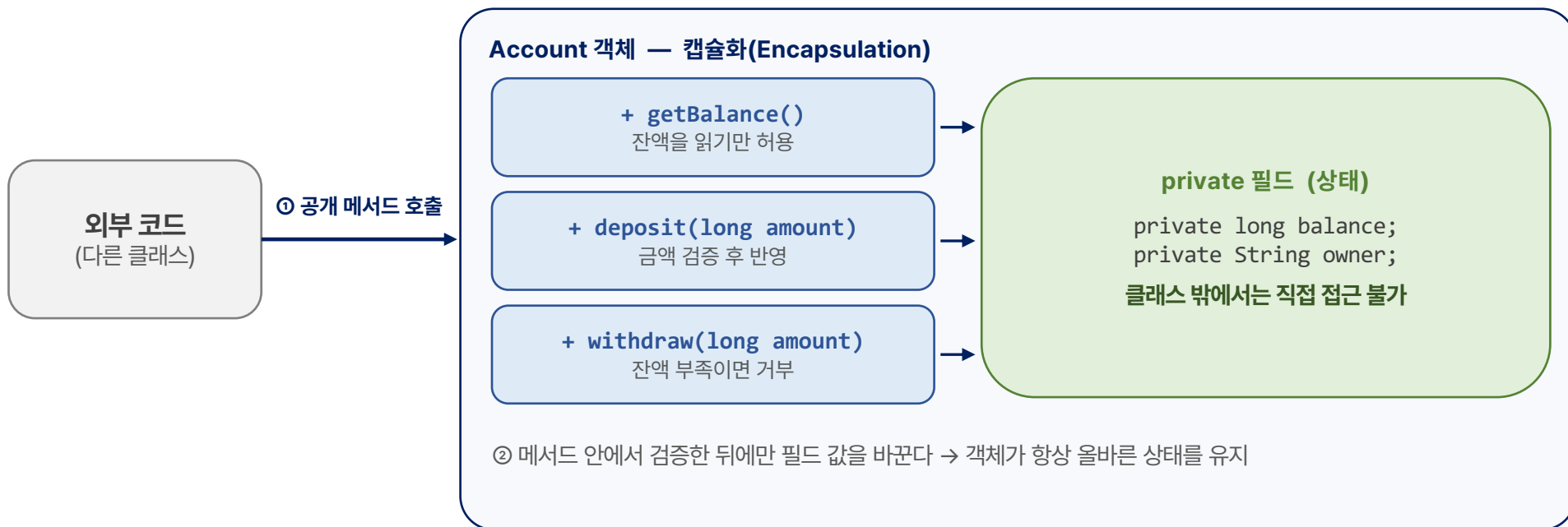
    public long getBalance() { return balance; }           // 읽기만 허용
    public void deposit(long amount) {                    // 검증 후 변경
        if (amount <= 0) throw new IllegalArgumentException("금액 오류");
        balance += amount;
    }
}
```

getter / setter 를 기계적으로 다 만들지 않는다

‘값을 바꾼다’가 아니라 ‘입금한다 · 취소한다’처럼 의미 있는 동작으로 공개한다.

[도식] 캡슐화 — private 필드와 공개 메서드

- 잘못된 값이 들어올 경로를 하나로 좁히는 것이 캡슐화의 목적이다



`account.balance = -1000;`
→ 컴파일 오류. private 필드는 클래스 밖에서 직접 읽거나 쓸 수 없다.

캡슐화 = 데이터를 숨기고(private), 검증된 통로(public 메서드)로만 접근하게 만드는 것

잘못된 값이 들어올 경로가 하나로 줄어들기 때문에, 버그가 생겼을 때 확인할 곳도 그 메서드 하나로 좁혀진다.

상속 (extends)

- 공통 코드를 부모 클래스에 두고 자식이 물려받는다. 자바는 클래스 다중 상속을 지원하지 않는다

상속 - 공통 코드를 부모에 두고 물려받는다

```
public class Vehicle {
    protected int speed;
    public void accelerate(int v) { speed += v; }
}

public class Car extends Vehicle {
    private int doors;
    // accelerate() 를 그대로 물려받는다
}

// 자바는 클래스 다중 상속을 지원하지 않는다
```

super - 부모에 접근

```
public class Car extends Vehicle {
    public Car(int doors) {
        super(); // 부모 생성자 (생략 시 자동 호출)
        this.doors = doors;
    }

    @Override
    public void accelerate(int v) {
        super.accelerate(v); // 부모 동작 + 추가 동작
        log(speed);
    }
}
```

상속은 '코드 재사용'이 아니라 '~는 ~이다(is-a)' 관계일 때만 쓴다

Car is a Vehicle 은 자연스럽지만, '메서드를 가져다 쓰고 싶어서' 상속하면 부모가 바뀔 때마다 자식이 깨진다. 그럴 땐 필드로 갖는 합성 (composition)이 낫다.

오버로딩과 오버라이딩

- 이름은 비슷하지만 완전히 다른 개념이다 — 하나는 '같은 이름의 다른 메서드', 하나는 '부모 메서드의 재정의'

	오버로딩 (Overloading)	오버라이딩 (Overriding)
무엇인가	같은 이름의 메서드를 여러 개 정의	부모가 만든 메서드를 자식이 다시 정의
구분 기준	파라미터의 개수 · 타입 · 순서	메서드 이름과 파라미터가 완전히 같아야 한다
반환 타입	반환 타입만 다른 것은 불가	부모와 같거나 그 하위 타입
결정 시점	컴파일 시점에 정해진다	실행 시점에 실제 객체를 보고 정해진다
접근 제어자	제약 없음	부모보다 좁힐 수 없다

오버로딩

```
void print(int i) { ... }
void print(String s) { ... }
void print(String s, int times) { ... }
```

오버라이딩 - @Override 를 꼭 붙인다

```
@Override
public String toString() { ... }
// 이름을 잘못 쓰면 컴파일 오류로 잡힌다
```

@Override 는 문서가 아니라 안전장치다

붙여 두면 '부모에 그런 메서드가 없다'는 실수를 컴파일러가 잡아 준다.

다형성 (Polymorphism)

- 부모(또는 인터페이스) 타입으로 받아 두면, 실제 주입된 객체에 따라 동작이 달라진다

다형성 - 부모(인터페이스) 타입으로 받아 자식의 동작을 실행한다

```
public interface Payment { void pay(int amount); }

public class CardPayment implements Payment { public void pay(int a) { /* 카드 승인 */ } }
public class KakaoPayment implements Payment { public void pay(int a) { /* 간편결제 */ } }

// 호출하는 쪽은 구현이 무엇인지 모른다
public class OrderService {
    private final Payment payment; // 인터페이스 타입으로 받는다
    public OrderService(Payment payment) { this.payment = payment; }

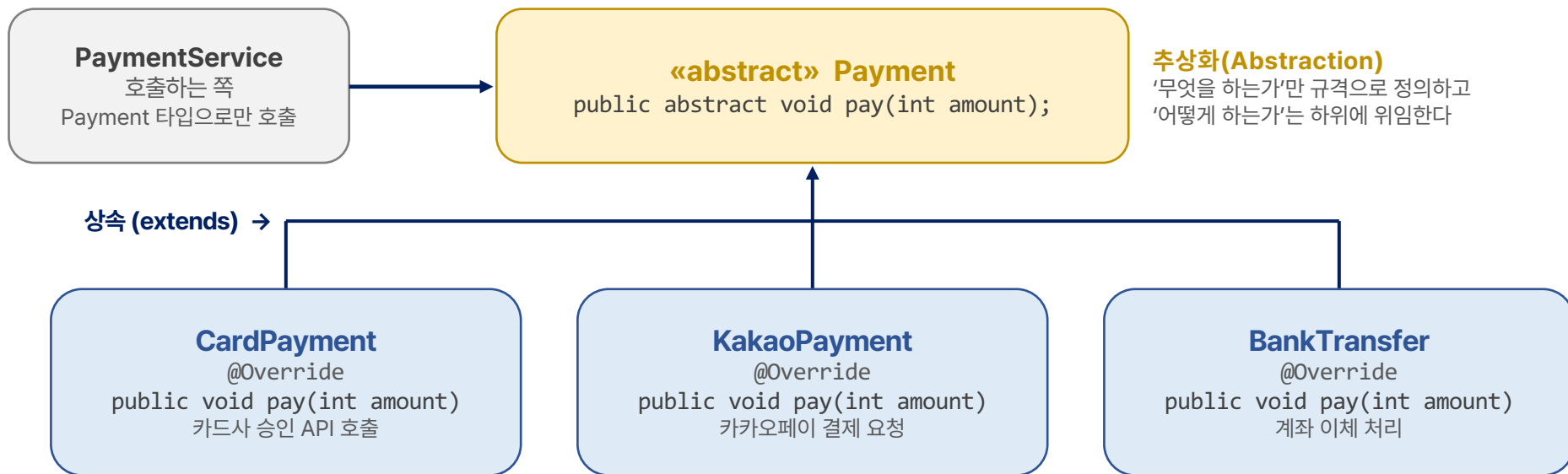
    public void order(int amount) { payment.pay(amount); } // 같은 호출, 다른 동작
}
```

결제 수단이 하나 늘어도 OrderService 는 고치지 않는다

새 클래스를 하나 추가하고 생성자에 넣어 주면 끝이다. '변경에는 닫히고 확장에는 열린다'는 OCP 가 바로 이것이고, 스프링의 의존성 주입(DI)은 이 구조를 프레임워크가 대신 연결해 주는 장치다.

[도식] 추상화 · 상속 · 다형성

- 공통 규격을 위에 두고, 구현을 물려받아 재사용하며, 같은 호출이 다르게 동작하게 한다



다형성(Polymorphism) : payment.pay(10000) 이라는 같은 호출이, 실제로 주입된 객체에 따라 서로 다르게 동작한다

상속 : 공통 코드를 부모에 두고 물려받는다 · 추상화 : 공통 규격만 남기고 구현은 미룬다 · 다형성 : 부모 타입으로 받아 자식의 동작을 실행한다

결제 수단이 하나 늘어도 **PaymentService** 는 고치지 않는다 — 새 클래스를 하나 추가하면 끝. 이것이 **SOLID** 의 **OC**P 로 이어진다.

추상 클래스 vs 인터페이스

- '~는 ~이다'를 물려줄 때는 추상 클래스, '~를 할 수 있다'를 규정할 때는 인터페이스

	추상 클래스 (abstract class)	인터페이스 (interface)
관계	'~는 ~이다' — 공통 조상	'~를 할 수 있다' — 역할·규격
다중 상속	하나만 상속할 수 있다	여러 개를 동시에 구현할 수 있다
필드	인스턴스 필드를 가질 수 있다	public static final 상수만 가능
생성자	있다 (자식이 super 로 호출)	없다
구현	구현 메서드 + 추상 메서드 혼합	default · static 메서드로 구현 제공
언제	상태와 공통 구현을 함께 물려줄 때	구현을 갈아 끼울 여지를 남길 때 (기본)

추상 클래스

```
public abstract class Shape {
    protected String name;
    public abstract double area();    // 구현 없음
}
```

인터페이스

```
public interface Shape {
    double area();                // 자동 public abstract
}
```

고민되면 인터페이스부터 — 스프링 코드가 인터페이스 중심인 이유

테스트에서 가짜 구현으로 바꿔 끼우기 쉽고, 구현을 교체해도 호출하는 쪽은 그대로다.

인터페이스의 default · static 메서드

- Java 8 이후 인터페이스도 구현을 가질 수 있게 됐다 — 이유를 알면 언제 쓸지도 보인다

인터페이스가 구현을 가질 수 있게 된 이유

```
public interface Notifier {  
    void send(String message);                // 구현체가 반드시 만들어야 하는 것  
  
    default void sendAll(List<String> messages) {    // 기본 구현 - 구현체가 안 만들어도 된다  
        messages.forEach(this::send);  
    }  
  
    static Notifier noop() { return msg -> { }; }    // 유틸 팩터리 - 구현체와 무관  
  
    private void log(String m) { ... }            // default 들끼리 공유하는 내부 헬퍼 (Java 9+)  
}
```

default 메서드는 '이미 배포된 인터페이스에 메서드를 추가하기 위해' 생겼다

Java 8 에서 Collection 에 stream() 을 넣어야 했는데, 그냥 추가하면 세상의 모든 구현체가 컴파일되지 않는다.

그래서 기본 구현을 함께 주는 방법이 필요했다. 남용하면 인터페이스가 추상 클래스처럼 무거워지니 최소한으로 쓴다.

static · final · enum

- static 은 객체가 아니라 클래스에 속하고, enum 은 '정해진 값의 집합'을 타입으로 만든다

static · final

```
public class Config {
    public static final int MAX = 100;    // 상수
    private static int count;             // 클래스당 하나

    public static int next() { return ++count; }
}

Config.MAX; Config.next();    // 객체 없이 호출

// static 메서드 안에서는 인스턴스 필드를 쓸 수 없다
// final : 변수는 재대입 금지 · 메서드는 재정의 금지
```

enum – 정해진 값의 집합

```
public enum OrderStatus {
    READY("준비"), PAID("결제완료"), DONE("배송완료");

    private final String label;
    OrderStatus(String label) { this.label = label; }
    public String label() { return label; }
}

OrderStatus.valueOf("PAID");    OrderStatus.values();
```

상태 값을 String 이나 int 로 들고 다니지 않는다

status.equals("PAID") 는 오타가 나도 컴파일러가 못 잡지만, enum 은 잡아 준다. switch 에서 모든 값을 다뤘는지도 확인해 준다.

Object 클래스 — equals · hashCode · toString

- 모든 클래스는 Object 를 상속한다. 값으로 비교하려면 equals 와 hashCode 를 함께 재정의한다

Object 의 메서드	왜 재정의하는가
toString()	로그·디버깅에서 객체를 사람이 읽을 수 있게. 기본값은 클래스명@해시코드
equals(Object o)	'같은 객체인가' 대신 '같은 값인가'로 비교하고 싶을 때
hashCode()	HashMap · HashSet 이 객체를 어느 칸에 넣을지 정할 때 쓴다
getClass()	실제 타입 확인 — 리플렉션·프레임워크가 많이 쓴다

equals 를 재정의하면 hashCode 도 반드시 함께 재정의한다

규약이 깨지면 HashSet 에 넣은 객체를 contains 로 찾지 못하는 일이 생긴다.
IDE 자동 생성 · Objects.hash(...) 를 쓰고, 값 객체라면 record 가 가장 안전하다.

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof User other)) return false;    // instanceof 패턴 (Java 16+)
    return age == other.age && Objects.equals(name, other.name);
}

@Override public int hashCode() { return Objects.hash(name, age); }
```

[핵심 요약] 객체지향 프로그래밍

캡슐화

필드는 private, 통로는 의미 있는 public 메서드
검증을 생성자와 메서드 안으로 모은다

상속

'~는 ~이다' 관계일 때만 — 아니면 합성
재정의에는 @Override 를 반드시 붙인다

추상화와 다형성

인터페이스로 규격만 정하고 구현은 같아 끼운다
같은 호출이 실제 객체에 따라 다르게 동작한다

Object 규약

equals 를 고치면 hashCode 도 함께
값 객체라면 record 가 가장 안전한 선택

스프링의 DI · AOP · JPA 는 모두 '인터페이스로 받고 구현을 같아 끼운다'는 이 원리 위에 서 있다

Java 21 모던 문법

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

record — 불변 데이터 캐리어

- '데이터를 나르는 것이 전부인 클래스'를 한 줄로 선언한다 (Java 16+)

예전 — 값 객체 하나에 60줄

```
public final class Point {
    private final int x;
    private final int y;
    public Point(int x, int y) { ... }
    public int x() { return x; }
    public int y() { return y; }
    @Override public boolean equals(Object o) { ... }
    @Override public int hashCode() { ... }
    @Override public String toString() { ... }
}
```

record (Java 16+) — 한 줄

```
public record Point(int x, int y) { }

// 컴파일러가 자동으로 만들어 주는 것
//   · private final 필드 x, y
//   · 전체 필드를 받는 생성자
//   · 접근자 x(), y() ← getX() 가 아니다
//   · equals / hashCode / toString

var p1 = new Point(1, 2);
p1.equals(new Point(1, 2)); // true
```

record 는 '이 클래스는 데이터를 나르는 것이 전부다'라는 선언이다

필드는 모두 final 이라 만들어진 뒤 바꿀 수 없고, 상속할 수도 상속받을 수도 없다. 대신 인터페이스는 구현할 수 있다.

record 활용과 주의점

- 검증과 파생 메서드는 넣을 수 있지만, 상태가 바뀌어야 하는 객체에는 쓰지 않는다

검증과 부가 메서드는 넣을 수 있다

```
public record Order(String id, int qty, long price) {

    public Order {                               // 압축 생성자
        if (qty <= 0)
            throw new IllegalArgumentException("수량 오류");
    }

    public long total() { return qty * price; }    // 파생 값

    static Order empty() { return new Order("", 1, 0); }
}
```

어디에 쓰나 / 어디에 쓰지 않나

- DTO – API 요청/응답 본문
- 조회 결과를 담는 값 객체
- 여러 값을 한 번에 돌려줄 때
- 패턴 매칭의 재료 (뒤에서 다룬다)
- × JPA 엔티티 – 기본 생성자와 가변 필드가 필요하다
- × 상태가 바뀌어야 하는 도메인 객체
- × 필드가 계속 늘어나는 설정 클래스

값을 바꾸고 싶으면 새 record 를 만든다

order.withQty(3) 같은 메서드를 직접 만들어 new Order(id, 3, price) 를 돌려주는 식이다. 불변이므로 여러 스레드가 같이 읽어도 안전하다.

sealed 클래스 · 인터페이스

- 상속할 수 있는 타입을 **permits** 로 지정해 계층을 봉인한다 (Java 17+)

sealed (Java 17+) – 상속할 수 있는 타입을 내가 지정한다

```
public sealed interface Shape permits Circle, Rectangle, Triangle { }

public record Circle(double radius)           implements Shape { }
public record Rectangle(double width, double h) implements Shape { }
public record Triangle(double base, double h)  implements Shape { }

// 봉인 계층에 참여하는 하위 타입은 셋 중 하나여야 한다
// final       : 더 이상 상속 불가 (record 는 자동으로 final)
// sealed      : 다시 봉인해 계층을 이어 간다
// non-sealed  : 여기서 봉인을 풀어 누구나 상속 가능
```

무엇이 좋아지나	설명
경우의 수가 닫힌다	Shape 의 구현이 셋뿐임을 컴파일러가 안다 → switch 가 default 없이 완결된다
설계 의도가 코드에 남는다	‘아무나 확장하라고 만든 타입이 아니다’를 주석이 아니라 문법으로 표현한다
빠뜨린 분기를 잡아 준다	타입을 하나 추가하면, 그 타입을 처리하지 않은 switch 가 전부 컴파일 오류

permits 를 생략할 수도 있다

하위 타입이 같은 파일 안에 있으면 permits 없이 sealed 만 붙여도 된다.

instanceof 패턴 매칭

- 타입을 검사하는 김에 변수에 담는다 — 캐스팅 한 줄이 사라진다 (Java 16+)

예전 — 검사하고, 캐스팅하고, 담는다

```
if (obj instanceof String) {
    String s = (String) obj;    // 같은 말을 두 번
    if (s.length() > 5) { ... }
}

// 타입을 세 번 적는다 : instanceof · 캐스트 · 선언
// 캐스팅을 빠뜨리면 컴파일 오류,
// 다른 타입으로 캐스팅하면 런타임 오류
```

instanceof 패턴 매칭 (Java 16+)

```
if (obj instanceof String s && s.length() > 5) {
    // s 를 바로 쓸 수 있다
}

// 검사에 성공하면 s 에 자동으로 담긴다
// s 의 유효 범위는 '검사가 참인 곳' 뿐이다

if (!(obj instanceof String s)) return;
// 여기서부터 s 를 쓸 수 있다 (early return)
```

가장 자주 쓰게 되는 자리 — equals

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof User other)) return false;    // 캐스팅 한 줄이 사라진다
    return age == other.age && Objects.equals(name, other.name);
}
```

패턴 매칭의 출발점

‘타입을 검사하는 김에 변수에 담는다’는 이 아이디어가 Java 21 의 switch 패턴 매칭 · record 패턴으로 확장된다.

switch 패턴 매칭 (Java 21)

- case 에 타입을 쓰고, when 으로 조건을 덧붙인다 — Java 21 에서 정식 기능이 됐다

예전 - if - else if 사다리

```
static String format(Object o) {
    if (o instanceof Integer i) {
        return "정수 " + i;
    } else if (o instanceof String s) {
        return "문자열 " + s;
    } else if (o instanceof Long l) {
        return "긴 정수 " + l;
    }
    return "알 수 없음";
}
```

switch 패턴 매칭 (Java 21 정식)

```
static String format(Object o) {
    return switch (o) {
        case Integer i -> "정수 " + i;
        case Long l -> "긴 정수 " + l;
        case String s -> "문자열 " + s;
        case null -> "값 없음"; // null 도 case 로
        default -> "알 수 없음";
    };
}
```

when 가드 - 타입 + 조건을 한 번에

```
case Integer i when i > 100 -> "큰 정수 " + i; // 조건을 덧붙이는 when 가드
case Integer i -> "정수 " + i; // 가드가 있는 case 를 먼저 적는다
```

case 의 순서가 의미를 가진다

먼저 맞는 것이 선택되므로 좁은 조건을 먼저 적는다. 앞 case 가 뒤를 가리면 컴파일 오류.

record 패턴 (Java 21)

- record 를 분해해 필드를 바로 변수로 받는다. 중첩 분해도 가능하다

record 패턴 (Java 21 정식) – 분해해서 받는다

```
record Point(int x, int y) { }
record Line(Point from, Point to) { }

// 타입만 확인하고 다시 꺼내던 방식
case Point pt -> pt.x() + pt.y();

// 분해해서 바로 변수로 받는다
case Point(int x, int y) -> x + y;

// 중첩 분해도 된다
case Line(Point(var x1, var y1), Point p2) -> ...
```

실제로 쓰는 모습

```
sealed interface Event permits Click, Scroll, Key { }
record Click(int x, int y) implements Event { }
record Scroll(int amount) implements Event { }
record Key(char c) implements Event { }

String describe(Event e) {
    return switch (e) {
        case Click(int x, int y) -> "클릭 " + x + "," + y;
        case Scroll(int amt) when amt > 0 -> "위로 스크롤";
        case Scroll(int amt) -> "아래로 스크롤";
        case Key(char c) -> "키 입력 " + c;
    }; // sealed 라 default 가 필요 없다
}
```

‘꺼내서 검사하고 다시 꺼내는’ 코드가 한 줄로 줄어든다

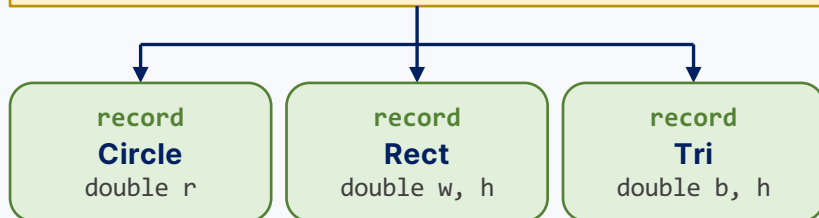
타입 확인 · 캐스팅 · 필드 추출 · 조건 검사가 case 한 줄 안에서 끝난다. 데이터를 record 로 모델링해 둘수록 이 이점이 커진다.

[도식] sealed + record + switch 조합

- 빠뜨린 분기를 런타임이 아니라 컴파일 시점에 잡아 주는 것이 이 조합의 핵심 가치다

① 타입 집합을 봉인한다 (sealed)

```
public sealed interface Shape
    permits Circle, Rect, Tri { }
```



permits 에 적힌 세 개가 전부다.

다른 곳에서 Shape 를 구현할 수 없으므로,
컴파일러가 '가능한 경우의 수'를 전부 안다.

② 그래서 switch 가 완결(exhaustive) 된다

```
double area(Shape s) {
    return switch (s) {
        case Circle(double r)          -> Math.PI * r * r;
        case Rect(double w, double h)  -> w * h;
        case Tri(double b, double h)   -> b * h / 2;
    }; // default 가 필요 없다
}
```

// 조건을 덧붙이려면 when 가드
case Circle c when c.r() > 100 -> ...

sealed 로 경우의 수를 닫고 · record 로 데이터를 열고 · switch 패턴 매칭으로 한 번에 분기한다

새 타입(예: Hexagon)을 permits 에 추가하면, 그 타입을 처리하지 않은 switch 는 전부 컴파일 오류가 난다.

'빠뜨린 분기'를 런타임이 아니라 컴파일 시점에 잡아 주는 것이 이 조합의 핵심 가치다.

[참고] Java 10 → 21 문법 개선 한눈에

- 21로 올린다는 것은 그 사이에 쌓인 문법 개선을 한꺼번에 쓸 수 있게 된다는 뜻이다

버전	기능	한 줄 정리
10	var	지역 변수의 타입을 컴파일러가 추론한다
14	switch 표현식	switch 가 값을 돌려주고, -> 는 fall-through 가 없다
15	텍스트 블록	""" ... """ 로 여러 줄 문자열을 보이는 그대로 쓴다
16	record · instanceof 패턴	불변 값 객체를 한 줄로 선언 · 타입 검사와 동시에 변수에 담기
17	sealed	상속할 수 있는 타입을 permits 로 지정해 계층을 봉인한다
21	switch · record 패턴 매칭	case 에 타입·조건을 쓰고, record 를 분해해 필드를 받는다 (정식)
21	Sequenced Collections	getFirst / getLast / reversed 를 표준 인터페이스로 제공
21	가상 스레드	수십만 개를 만들 수 있는 경량 스레드 (정식)

미리보기(preview) 기능은 실무 코드에 넣지 않는다

구조적 동시성 · Scoped Values 는 미리보기다. --enable-preview 로만 켜지고 바뀔 수 있다

(21·22 에 미리보기로 있던 문자열 템플릿은 실제로 철회됐다).

[핵심 요약] Java 21 모던 문법

record

불변 값 객체를 한 줄로 — 생성자·접근자·equals·hashCode 자
동
DTO 와 조회 결과에 우선 적용, JPA 엔티티에는 쓰지 않는다

sealed

상속 가능한 타입을 permits 로 못 박는다
경우의 수가 닫히면 컴파일러가 빠진 분기를 잡아 준다

패턴 매칭

instanceof → switch → record 로 확장됐다
타입 확인 · 캐스팅 · 필드 추출이 한 줄로

when 가드

타입과 조건을 case 한 줄에 함께 적는다
좁은 조건을 먼저 — 순서가 의미를 가진다

sealed 로 경우의 수를 닫고 · record 로 데이터를 열고 · switch 패턴 매칭으로 분기한다 — Java 21 문법의 핵심 조합

예외 처리

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

예외란 무엇인가

- 예외는 '정상 흐름에서 벗어났다'는 신호다. 처리하지 않으면 스레드가 그대로 종료된다

예외를 처리하지 않으면

```
public static void main(String[] args) {
    int[] a = new int[3];
    a[5] = 1;        // 여기서 예외 발생
    System.out.println("여기는 실행되지 않는다");
}
```

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 3
```

try-catch 로 흐름을 이어받는다

```
try {
    int[] a = new int[3];
    a[5] = 1;
} catch (ArrayIndexOutOfBoundsException e) {
    log.warn("범위를 벗어남", e);
}
System.out.println("여기는 실행된다");

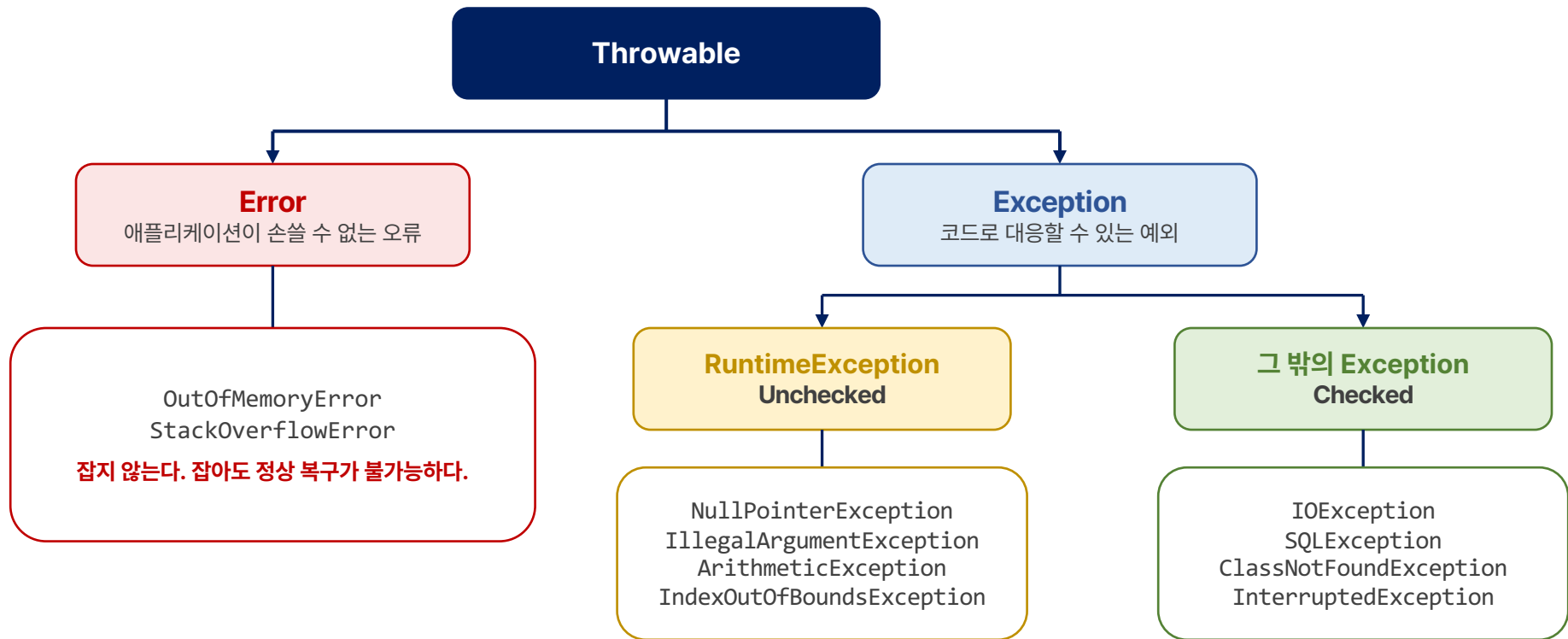
// 예외 객체 e 에는 메시지와 스택 트레이스가 들어 있다
```

예외는 '프로그램이 죽는 사건'이 아니라 '정상 흐름에서 벗어났다는 신호'다

그 신호를 어디에서 받아 어떻게 대응할지 정하는 것이 예외 처리다. 신호를 잡아서 그냥 버리는 것(빈 catch)이 가장 나쁜 처리다.

[도식] Java 예외 계층도

- 모든 예외의 조상은 Throwable 이고, 그 아래가 Error 와 Exception 으로 갈린다



Checked (컴파일러가 강제) : try-catch 로 잡거나 throws 로 넘긴다고 선언해야 컴파일된다 — 외부 자원처럼 '예상 가능한 실패'

Unchecked (RuntimeException 계열) : 선언하지 않아도 된다 — 대개 코드의 버그이므로, 잡아서 감추기보다 원인을 고친다

스프링은 데이터 접근 예외를 모두 **Unchecked** 로 감싸 서비스 코드가 try-catch 로 지저분해지지 않게 한다.

Checked vs Unchecked

- 컴파일러가 처리 여부를 강제하느냐로 갈린다 — 이 차이가 코드 구조를 크게 바꾼다

	Checked 예외	Unchecked 예외 (RuntimeException 계열)
컴파일러	잡거나 throws 로 선언해야 컴파일된다	아무 선언도 요구하지 않는다
의미	예상 가능한 실패 — 외부 자원·입력	대개 코드의 버그
예	IOException · SQLException	NullPointerException · IllegalArgumentException
대응	재시도 · 대체 경로 · 사용자 안내	감추지 말고 원인을 고친다

```
// Checked — 선언하지 않으면 컴파일되지 않는다
public String read(Path p) throws IOException { return Files.readString(p); }

// Unchecked — 선언 없이 던질 수 있다
public void setAge(int age) { if (age < 0) throw new IllegalArgumentException("나이 오류"); }
```

스프링이 데이터 접근 예외를 모두 Unchecked 로 감싸는 이유

SQLException 은 Checked 라 서비스·컨트롤러까지 throws 가 번져 나간다.

스프링은 이를 Unchecked 로 바꿔 비즈니스 코드를 깨끗하게 유지하고, 처리는 한 곳에 모은다.

try-catch-finally 와 multi-catch

- 구체적인 예외를 먼저 잡고, 넓은 예외를 나중에 잡는다

try - catch - finally

```
try {
    process();
} catch (IllegalArgumentException e) {
    // 구체적인 예외를 먼저
} catch (Exception e) {
    // 넓은 예외는 나중에 (순서를 바꾸면 컴파일 오류)
} finally {
    // 예외가 나든 안 나든 반드시 실행
}
```

multi-catch – 같은 처리를 묶는다

```
try {
    parseAndSave(input);
} catch (NumberFormatException | DateTimeParseException e) {
    throw new InvalidInputException("입력 형식 오류", e);
}

// 원인 예외(e)를 반드시 넘겨준다
// 넘기지 않으면 진짜 원인이 스택 트레이스에서 사라진다
```

하지 말아야 할 세 가지

- ① catch (Exception e) {} — 빈 catch. 문제가 조용히 사라져 원인을 찾을 수 없다
- ② e.printStackTrace() — 로그 설정을 타지 않는다. log.error("...", e) 를 쓴다
- ③ finally 안에서 return — 원래 예외를 삼켜 버린다

throw · throws · 사용자 정의 예외

- 상황에 맞는 이름의 예외를 만들어 던지면, 잡는 쪽이 대응을 나눌 수 있다

throw · throws · 사용자 정의 예외

```
// 1) 사용자 정의 예외 - 무엇이 잘못됐는지 이름으로 말한다
public class OrderNotFoundException extends RuntimeException {
    private final String orderId;
    public OrderNotFoundException(String orderId) {
        super("주문을 찾을 수 없습니다: " + orderId);
        this.orderId = orderId;
    }
    public String getOrderId() { return orderId; }
}

// 2) 던지기
public Order find(String id) {
    return repository.findById(id).orElseThrow(() -> new OrderNotFoundException(id));
}
```

키워드	의미
throw	지금 이 자리에서 예외 객체를 던진다 (문장)
throws	이 메서드는 이런 예외를 밖으로 넘길 수 있다는 선언 (메서드 시그니처)

예외 이름 자체가 문서다

RuntimeException 을 그대로 던지면 잡는 쪽이 구분할 수 없다. 상황별 타입을 만들어 둔다.

try-with-resources

- 파일 · 소켓 · DB 커넥션처럼 반드시 닫아야 하는 자원을 자동으로 닫아 준다

finally 로 닫기 – 장황하고 실수하기 쉽다

```
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader(path));
    return br.readLine();
} catch (IOException e) {
    throw new UncheckedIOException(e);
} finally {
    if (br != null) {
        try { br.close(); } catch (IOException ignored) { }
    }
}
```

try-with-resources (Java 7+)

```
try (var br = new BufferedReader(new FileReader(path))) {
    return br.readLine();
} catch (IOException e) {
    throw new UncheckedIOException(e);
}

// 괄호 안에서 연 자원은 블록을 벗어날 때 자동으로 닫힌다
// 여러 개를 세미콜론으로 나열할 수 있고,
// 나중에 연 것부터 역순으로 닫힌다

// 조건 : AutoCloseable 을 구현한 타입
```

파일 · 소켓 · DB 커넥션은 반드시 닫아야 한다

닫지 않으면 파일 핸들과 커넥션이 쌓여 결국 서비스가 멈춘다. 자바에서 자원을 다룰 때는 try-with-resources 를 기본으로 삼는다.

[핵심 요약] 예외 처리

계층

Throwable → Error / Exception
Error 는 잡지 않는다 · Exception 은 대응한다

Checked vs Unchecked

Checked 는 컴파일러가 처리 여부를 강제한다
Unchecked 는 대개 버그 — 감추지 말고 고친다

잡을 때

구체적인 예외를 먼저, 넓은 예외를 나중에
빈 catch 금지 · 원인 예외(e)를 반드시 함께 넘긴다

자원 관리

파일·커넥션은 try-with-resources 로 연다
AutoCloseable 이면 자동으로, 역순으로 닫힌다

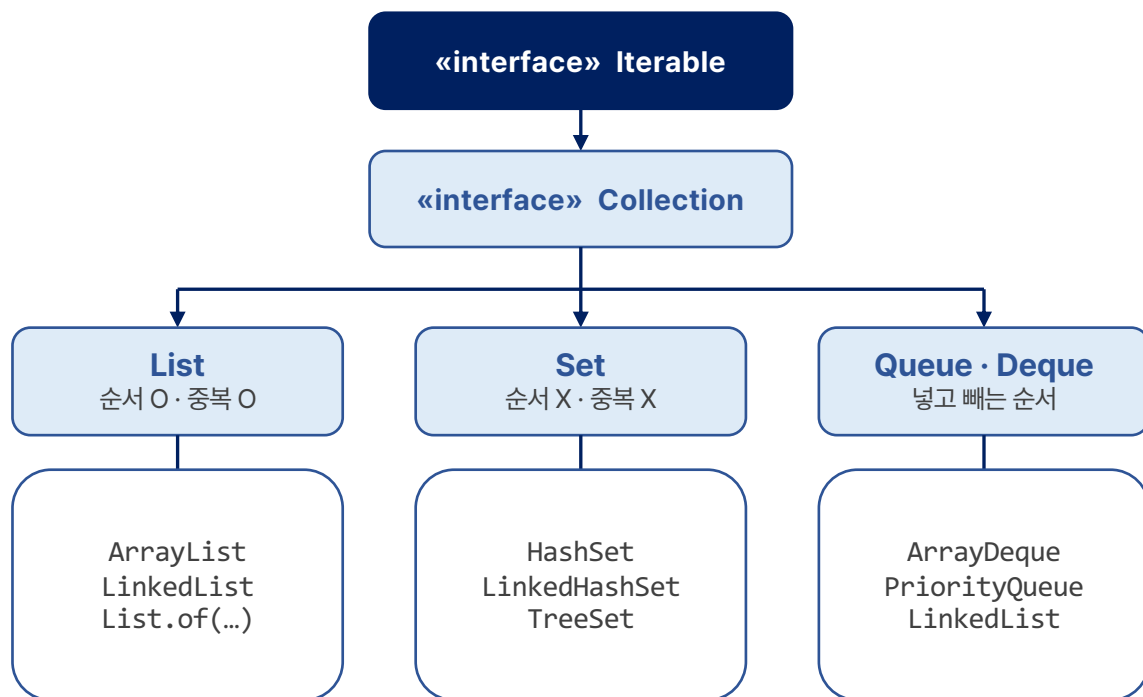
예외를 어디서 잡을지 정하는 기준 : '여기서 의미 있게 대응할 수 있는가' — 아니라면 그대로 위로 넘긴다

컬렉션과 제네릭

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

[도식] 컬렉션 프레임워크 계층도

- Collection 계열(List · Set · Queue)과 Map 계열은 서로 다른 계층이다



Map — Collection 을 상속하지 않는 별도 계층

«interface» Map<K, V>

HashMap
LinkedHashMap
TreeMap
Map.of(...) — 불변

키는 중복될 수 없고, 값은 중복될 수 있다.
키 객체는 equals / hashCode 가
올바르게 구현돼 있어야 한다.

Java 21 — Sequenced Collections (JEP 431)

List · LinkedHashSet · LinkedHashMap 처럼 '순서가 있는' 컬렉션에 공통 인터페이스가 생겼다.

getFirst() · getLast() · addFirst() · addLast() · removeFirst() · removeLast() · reversed()

→ list.get(0) · list.get(size - 1) · iterator().next() 같은 관용구가 사라진다

List — ArrayList 와 LinkedList

- 순서가 있고 중복을 허용하는, 가장 많이 쓰는 컬렉션

자주 쓰는 조작

```
List<String> list = new ArrayList<>();
list.add("A"); list.add(0, "B"); // 끝 / 지정 위치
list.get(0); list.set(0, "C");
list.remove("A"); // 값으로 삭제
list.remove(0); // 인덱스로 삭제 (주의: 오버로딩)
list.contains("C"); list.indexOf("C"); list.size();

list.removeIf(s -> s.startsWith("A")); // 조건 삭제
list.sort(Comparator.naturalOrder());
```

ArrayList vs LinkedList

ArrayList - 내부가 배열

- 인덱스 조회가 빠르다 $O(1)$
- 중간 삽입·삭제는 뒤를 밀어야 한다
- 사실상 기본 선택

LinkedList - 노드가 앞뒤로 연결

- 앞뒤 추가·삭제가 빠르다
- 인덱스 조회는 처음부터 훑는다 $O(n)$
- 큐/덱 용도라면 ArrayDeque 가 더 낫다

list.remove(0) 과 list.remove(Integer.valueOf(0)) 은 다르다

List<Integer> 에서 remove(int) 는 인덱스, remove(Object) 는 값이다. 정수 리스트에서 값을 지우려면 Integer.valueOf 로 감싸야 한다.

Set — HashSet · LinkedHashSet · TreeSet

- 중복을 허용하지 않는다. 어떤 순서를 원하느냐로 구현을 고른다

구현	순서	성능	언제 쓰나
HashSet	보장하지 않음	추가·조회·삭제 평균 $O(1)$	중복 제거만 필요할 때 (기본 선택)
LinkedHashSet	넣은 순서 유지	HashSet + 약간의 메모리	중복은 없애되 입력 순서를 지키고 싶을 때
TreeSet	정렬된 순서	$O(\log n)$	항상 정렬 상태로 두거나 범위 조회가 필요할 때

예

```
Set<String> tags = new HashSet<>(List.of("java", "spring", "java"));
tags.size();
// 2 - 중복이 사라진다

// 집합 연산
a.retainAll(b); // 교집합      a.addAll(b); // 합집합      a.removeAll(b); // 차집합

// TreeSet 의 범위 조회
TreeSet<Integer> scores = new TreeSet<>(List.of(10, 30, 50, 70));
scores.first(); scores.last(); scores.headSet(50); scores.tailSet(30);
```

HashSet 에 넣을 객체는 equals 와 hashCode 가 올바르게 구현되어야 한다

둘 중 하나만 재정의하면 '넣었는데 contains 가 false'가 된다. 값 객체는 record 가 안전하다.

Map — HashMap · TreeMap

- 키로 값을 찾는다. 키는 중복될 수 없고, 값은 중복될 수 있다

기본 조작

```
Map<String, Integer> stock = new HashMap<>();
stock.put("사과", 10);
stock.get("사과");           // 10, 없으면 null
stock.getDefault("배", 0);   // 없으면 기본값
stock.containsKey("사과");
stock.remove("사과");

for (var e : stock.entrySet()) {
    e.getKey(); e.getValue();
}
```

알아 두면 코드가 짧아지는 메서드

```
stock.putIfAbsent("배", 5);           // 없을 때만 넣는다
stock.computeIfAbsent("배", k -> new ArrayList<>());
stock.merge("사과", 1, Integer::sum); // 합치거나 넣기

stock.forEach((k, v) -> log.info("{}={}", k, v));

// 정렬이 필요하면 TreeMap,
// 넣은 순서가 필요하면 LinkedHashMap
```

get() 이 null 을 돌려줄 수 있다는 것을 항상 의식한다

getOrDefault · computeIfAbsent 를 쓰면 null 검사 코드가 사라진다. 카운팅에는 merge(key, 1, Integer::sum) 이 관용구다.

정렬 — Comparable 과 Comparator

- 타입의 기본 순서는 Comparable, 상황별 순서는 Comparator 로 정한다

Comparable – 그 타입의 '기본 정렬 순서'

```
public record User(String name, int age)
    implements Comparable<User> {

    @Override
    public int compareTo(User o) {
        return Integer.compare(this.age, o.age);
    }
}

Collections.sort(users); // 나이 오름차순
// 음수 : 내가 앞 / 0 : 같음 / 양수 : 내가 뒤
```

Comparator – 그때그때 다른 기준

```
users.sort(Comparator.comparing(User::name));

users.sort(Comparator
    .comparingInt(User::age).reversed() // 나이 내림차순
    .thenComparing(User::name));       // 같으면 이름순

// null 이 섞여 있다면
Comparator.nullsFirst(Comparator.comparing(User::name));

// 스트림에서도 그대로 쓴다
users.stream().sorted(Comparator.comparing(User::name))
```

빨셈으로 비교하지 않는다

return a.age - b.age 는 값이 클 때 오버플로로 부호가 뒤집힌다. Integer.compare(a, b) 를 쓴다.

제네릭 기초

- 타입을 파라미터로 받아, 캐스팅 없이 컴파일 시점에 타입 오류를 잡는다

제네릭이 없다면

```
List list = new ArrayList(); // 아무거나 들어간다
list.add("문자열");
list.add(42);

String s = (String) list.get(1); // 컴파일은 통과
// 실행 중 ClassCastException

// 타입 오류를 런타임까지 미루는 구조
```

제네릭 - 타입을 파라미터로 받는다

```
List<String> list = new ArrayList<>();
list.add("문자열");
// list.add(42); ← 컴파일 오류로 즉시 잡힌다
String s = list.get(0); // 캐스팅 불필요

// 직접 만들기
public class Box<T> {
    private T value;
    public T get() { return value; }
}

public <T extends Number> double sum(List<T> list) { ... }
```

제네릭 정보는 컴파일 후 지워진다 (타입 소거)

그래서 new T() 나 T[] 를 만들 수 없고, List<String> 과 List<Integer> 는 실행 시점에 같은 List 다.

즉 제네릭은 '실행을 빠르게 하는 장치'가 아니라 '컴파일러에게 타입을 알려 주어 실수를 막는 장치'다.

제네릭 와일드카드

- `List<Integer>` 는 `List<Number>` 가 아니다 — 그 간극을 메우는 장치가 와일드카드다

표기	이름	의미와 쓰임
<code><?></code>	비한정 와일드카드	타입을 신경 쓰지 않는다 — 읽기만 하고 요소 타입이 무관할 때
<code><? extends T></code>	상한 (upper bound)	T 또는 그 하위 타입 — 값을 꺼내 읽는 용도(Producer)
<code><? super T></code>	하한 (lower bound)	T 또는 그 상위 타입 — 값을 넣는 용도(Consumer)

왜 필요한가

```
// List<Number> 자리에 List<Integer> 를 넘길 수 없다 - 제네릭은 상속 관계를 물려받지 않는다
void printAll(List<Number> list) { ... }
printAll(intList);           // 컴파일 오류

void printAll(List<? extends Number> list) { ... } // 읽기 전용으로 받으면 가능
printAll(intList);           // OK - 단, list.add(...) 는 할 수 없다

void addAll(List<? super Integer> list) { list.add(1); } // 넣을 수 있다
```

PECS — Producer extends, Consumer super

값을 꺼내 쓰는(생산자) 파라미터에는 `extends`, 값을 넣는(소비자) 파라미터에는 `super` 를 쓴다. 둘 다 해야 한다면 와일드카드 없이 `<T>` 를 쓴다.

불변 컬렉션과 Sequenced Collections

- List.of 로 못 바꾸는 컬렉션을 만들고, Java 21 부터는 앞/뒤 접근이 표준화됐다

불변 컬렉션 (Java 9+)

```
List<String> l = List.of("a", "b", "c");
Set<String> s = Set.of("a", "b");
Map<String, Integer> m = Map.of("a", 1, "b", 2);

// l.add("d"); → UnsupportedOperationException

// 기존 컬렉션으로부터 복사본 만들기
List<String> copy = List.copyOf(mutableList);

// 반환값으로 내보낼 때 유용 - 밖에서 못 바꾼다
```

Sequenced Collections (Java 21)

```
// 예전
list.get(0);                list.get(list.size() - 1);
linkedHashSet.iterator().next(); // 첫 원소를 꺼내려면

// Java 21
list.getFirst();            list.getLast();
list.addFirst(x);           list.removeLast();
list.reversed();            // 역순 뷰 (복사 아님)

linkedHashMap.firstEntry(); linkedHashMap.putFirst(k, v);
```

새 인터페이스	적용 대상
SequencedCollection	List · Deque — getFirst / getLast / addFirst / addLast / removeFirst / removeLast / reversed
SequencedSet	LinkedHashSet · TreeSet(SortedSet)
SequencedMap	LinkedHashMap · TreeMap — firstEntry / lastEntry / putFirst / putLast / reversed

[핵심 요약] 컬렉션과 제네릭

무엇을 고를까

순서·중복 O → List / 중복 X → Set / 키-값 → Map
특별한 이유가 없으면 ArrayList · HashSet · HashMap

해시 기반의 전제

키·원소는 equals 와 hashCode 가 올바르게 동작해야 한다
값 객체라면 record 가 가장 안전하다

제네릭

타입 오류를 런타임이 아니라 컴파일에 잡는다
PECS — 꺼내면 extends, 넣으면 super

Java 21

Sequenced Collections 로 앞/뒤 접근이 표준화됐다
get(size-1) · iterator().next() 관용구가 사라진다

컬렉션 선택은 성능보다 '의도' 먼저 — 중복을 허용할 것인가, 순서가 의미가 있는가, 키로 찾을 것인가

함수형 프로그래밍

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

람다식

▪ '무엇을 할지'만 남기고 껍데기를 걷어낸 문법 — 익명 클래스의 축약형

익명 클래스 – 의도 한 줄에 껍데기 다섯 줄

```
Collections.sort(users, new Comparator<User>() {
    @Override
    public int compare(User a, User b) {
        return a.name().compareTo(b.name());
    }
});

new Thread(new Runnable() {
    @Override public void run() { print("실행"); }
}).start();
```

람다식 – 몸통만 남긴다

```
users.sort((a, b) -> a.name().compareTo(b.name()));

new Thread(() -> System.out.println("실행")).start();

// 형태
(파라미터) -> 식 // 값을 그대로 돌려준다
(파라미터) -> { 문장; return 값; }

• 파라미터 타입은 생략 가능 (추론)
• 파라미터가 하나면 괄호도 생략 가능
```

람다는 '추상 메서드가 딱 하나인 인터페이스' 자리에만 들어갈 수 있다

그래서 Comparator · Runnable 자리에는 되고, 아무 인터페이스에나 되는 것은 아니다. 이런 인터페이스를 함수형 인터페이스라고 부른다.

함수형 인터페이스

- 추상 메서드가 딱 하나인 인터페이스 — 람다가 들어갈 수 있는 자리다

인터페이스	추상 메서드	의미	쓰는 곳
Function<T,R>	R apply(T t)	받아서 변환	stream().map(...)
Consumer<T>	void accept(T t)	받아서 소비	forEach(...) · ifPresent(...)
Supplier<T>	T get()	요청하면 만들어 줌	orElseGet(...) · 지연 생성
Predicate<T>	boolean test(T t)	참/거짓 판정	filter(...) · removelf(...)
BiFunction<T,U,R>	R apply(T,U)	둘 받아 변환	map.merge(...) · reduce(...)

직접 만들 수도 있다

```
@FunctionalInterface                // 추상 메서드가 둘 이상이면 컴파일 오류로 잡아 준다
public interface Validator<T> {
    boolean validate(T target);
}

Validator<String> notBlank = s -> s != null && !s.isBlank();
```

표준 인터페이스로 충분한지 먼저 본다

java.util.function 에 40여 개가 있다. 이름이 의미를 드러낼 때만 직접 만든다.

메서드 참조

- 람다가 받은 값을 그대로 넘기기만 한다면 :: 로 더 줄일 수 있다

형태	메서드 참조	같은 뜻의 람다
정적 메서드	Integer.parseInt	s -> Integer.parseInt(s)
특정 객체의 메서드	System.out.println	s -> System.out.println(s)
임의 객체의 메서드	String.toUpperCase	s -> s.toUpperCase()
생성자	ArrayList.new	() -> new ArrayList<>()

줄일 수 있을 때만 줄인다

// 람다가 '받은 것을 그대로 넘기기만' 한다면 메서드 참조로 줄일 수 있다

```
names.forEach(s -> System.out.println(s));      → names.forEach(System.out::println);
list.stream().map(u -> u.name())                 → list.stream().map(User::name)
strs.stream().map(s -> Integer.parseInt(s))      → strs.stream().map(Integer::parseInt)
```

// 값을 가공하거나 인자 순서를 바꾸면 메서드 참조로 줄일 수 없다

```
list.stream().map(u -> u.name().toUpperCase()) // 그대로 람다로 둔다
```

짧게 쓰는 것이 목적이 아니라 '무엇을 하는지'가 드러나는 것이 목적이다

User::name 은 '이름을 꺼낸다'가 바로 보이지만, 억지로 줄인 메서드 참조는 오히려 읽기 어렵다.

[도식] 스트림 파이프라인 구조

- 스트림은 데이터를 담는 그릇이 아니라 데이터가 지나가는 통로다

```
users.stream().filter(u -> u.age() >= 20).map(User::name).sorted().toList()
```



스트림은 데이터를 담는 그릇이 아니라 '데이터가 지나가는 통로'다 — 원본 컬렉션은 그대로 남는다.

최종 연산이 없으면 중간 연산은 실행되지 않는다. 그리고 한 번 소비한 스트림은 다시 쓸 수 없다.

스트림 생성과 중간 연산

- 중간 연산은 게으르다 — 최종 연산을 만나기 전까지 아무 일도 일어나지 않는다

스트림 만들기

```
list.stream(); // 컬렉션
Stream.of("a", "b", "c");
Arrays.stream(array);
IntStream.range(0, 10); // 0..9
IntStream.rangeClosed(1, 10); // 1..10
Files.lines(path); // 파일 한 줄씩
Stream.iterate(1, i -> i * 2).limit(10);

// 원시 타입 스트림은 박싱을 피한다
// IntStream · LongStream · DoubleStream
```

중간 연산 – 몇 개든 이어 붙인다

```
filter(u -> u.age() >= 20) // 조건에 맞는 것만
map(User::name) // 다른 값으로 변환
flatMap(o -> o.items().stream()) // 중첩 구조를 펼친다
distinct() // 중복 제거 (equals 기준)
sorted(comparator) // 정렬
limit(10) / skip(5) // 앞에서 자르기 / 건너뛰기
peek(System.out::println) // 디버깅용 엿보기

// 모두 새 스트림을 돌려준다 (원본은 그대로)
// 최종 연산이 없으면 실행되지 않는다
```

map 과 flatMap 의 차이

map 은 원소 하나를 값 하나로 바꾼다(1:1). flatMap 은 원소 하나가 여러 값이 될 때(1:N) 그 결과를 한 줄로 펼쳐 준다.

예: 주문 목록 → 각 주문의 상품 목록 → 전체 상품 하나의 스트림.

스트림 최종 연산

- 최종 연산은 하나만 올 수 있고, 그 순간 파이프라인 전체가 실행된다

최종 연산	결과	설명
<code>toList()</code>	<code>List<T></code>	Java 16+ · 불변 리스트를 돌려준다 (가장 많이 쓴다)
<code>collect(...)</code>	다양	Collectors 로 원하는 모양을 지정한다
<code>forEach(...)</code>	없음	각 원소에 부수효과 — 순서가 필요하면 <code>forEachOrdered</code>
<code>count()</code> / <code>sum()</code>	long	개수 / 합계 (sum 은 원시 스트림에서)
<code>anyMatch</code> · <code>allMatch</code>	boolean	있는가 / 모두인가 — 찾으면 즉시 멈춘다
<code>findFirst</code> · <code>findAny</code>	<code>Optional<T></code>	첫 번째 / 아무거나 — 없을 수 있으므로 Optional
<code>reduce(...)</code>	<code>Optional<T></code> · <code>T</code>	두 개씩 접어 하나로 만든다 (합계·최댓값 등)
<code>min</code> · <code>max</code>	<code>Optional<T></code>	Comparator 기준 최소 · 최대

```

long adults = users.stream().filter(u -> u.age() >= 20).count();
int total   = orders.stream().mapToInt(Order::amount).sum();      // 원시 스트림으로 바꿔 합산
Optional<User> oldest = users.stream().max(Comparator.comparingInt(User::age));

```

Collectors

- 결과를 어떤 모양으로 모을지 지정한다 — 리스트 · 문자열 · 그룹 · 맵

자주 쓰는 Collectors

```
// 모아 담기
List<String> names = users.stream().map(User::name).collect(Collectors.toList());
Set<String> set = users.stream().map(User::name).collect(Collectors.toSet());

// 이어 붙이기
String line = users.stream().map(User::name).collect(Collectors.joining(", ", "[", "]"));

// 그룹으로 나누기 → Map<K, List<T>>
Map<String, List<User>> byCity = users.stream().collect(Collectors.groupingBy(User::city));

// 그룹별 집계
Map<String, Long> countByCity = users.stream()
    .collect(Collectors.groupingBy(User::city, Collectors.counting()));

// 참/거짓 두 덩이로 나누기 → Map<Boolean, List<T>>
Map<Boolean, List<User>> adult = users.stream().collect(Collectors.partitioningBy(u -> u.age() >= 20));

// 키-값 맵 만들기 (키가 겹치면 세 번째 인자로 병합 규칙을 준다)
Map<String, Integer> ageByName = users.stream().collect(Collectors.toMap(User::name, User::age, (a, b) -> a));
```

toMap 은 키가 겹치면 IllegalStateException 을 던진다

키 중복 가능성이 조금이라도 있으면 병합 함수를 반드시 넘긴다. 예상치 못한 운영 장애의 단골 원인이다.

Optional

- '값이 없을 수 있음'을 타입으로 드러내, null 검사를 흐름 안으로 넣는다

만들기과 꺼내기

```
Optional<User> o = repository.findById(id);

o.isPresent();           // 있는가
o.get();                  // 없으면 예외 (지양)
o.orElse(defaultUser);    // 없으면 이 값
o.orElseGet(() -> load()); // 없을 때만 계산
o.orElseThrow(() -> new NotFound(id));

Optional.of(v);           // v 가 null 이면 예외
Optional.ofNullable(v);   // null 이면 비어 있는 Optional
```

체이닝 - if 문이 사라진다

```
// 예전
User u = repository.findById(id);
if (u != null && u.address() != null) {
    return u.address().city();
}
return "미상";

// Optional
return repository.findById(id)
    .map(User::address)
    .map(Address::city)
    .orElse("미상");
```

Optional 은 '값이 없을 수 있음'을 타입으로 알리는 장치다

메서드 반환 타입에 쓰는 것이 본래 용도다. 필드 · 파라미터 · 컬렉션 원소에는 쓰지 않는다 — 직렬화도 안 되고 코드만 복잡해진다.

[참고] 스트림 · Optional 안티패턴

- 잘못 쓰면 오히려 읽기 어려워진다 — 자주 보이는 여섯 가지

이렇게 쓰면	이렇게 바꾼다
<code>o.isPresent()</code> 로 검사하고 <code>o.get()</code> 으로 꺼낸다	<code>map / orElse / orElseThrow</code> 로 체이닝한다
Optional 을 필드나 파라미터 타입으로 쓴다	반환 타입에만 쓴다 — 파라미터는 오버로딩으로 푼다
<code>stream().forEach()</code> 안에서 리스트에 add 한다	<code>collect(...)</code> 또는 <code>toList()</code> 로 모은다
스트림 하나를 두 번 사용한다	매번 새로 만든다 — 소비된 스트림은 재사용 불가
단순 반복까지 전부 스트림으로 바꾼다	부수효과가 목적이면 for-each 가 더 읽기 쉽다
습관적으로 <code>parallelStream()</code> 을 붙인다	데이터가 아주 크고 연산이 무거울 때만 — 대개 더 느려진다

스트림은 '무엇을 하는지'가 드러날 때 쓴다

`filter` → `map` → `collect` 처럼 데이터를 변형해 새 결과를 만드는 흐름에 강하다. 반대로 조건 분기가 많고 부수효과가 목적이면 일반 반복문이 낫다.

[핵심 요약] 함수형 프로그래밍

람다와 함수형 인터페이스

추상 메서드가 하나인 인터페이스 자리에만 들어간다
Function · Consumer · Supplier · Predicate

스트림 3단계

생성 → 중간 연산(lazy) → 최종 연산(여기서 실행)
원본은 바뀌지 않고, 한 번 쓴 스트림은 재사용 불가

Collectors

toList · joining · groupingBy · partitioningBy
toMap 은 키 중복 대비 병합 함수를 꼭 넘긴다

Optional

'값이 없을 수 있음'을 반환 타입으로 알린다
isPresent + get 대신 map · orElseThrow 체이닝

스트림과 Optional 의 목적은 짧게 쓰는 것이 아니라, '어떻게' 대신 '무엇을' 드러내는 것이다

동시성과 가상 스펙트럼

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

프로세스와 스레드

- 스레드는 힙을 공유하고 스택은 각자 가진다 — 공유하기 때문에 문제가 생긴다

	프로세스	스레드
단위	실행 중인 프로그램 하나	프로세스 안에서 동시에 도는 작업 흐름
메모리	서로 완전히 분리되어 있다	힙·메서드 영역을 공유하고 스택은 각자 가진다
통신	IPC 등 별도 수단이 필요하다	같은 객체를 그냥 참조하면 된다
대가	생성 비용이 크다	공유하기 때문에 동기화 문제가 생긴다

```
// 웹 애플리케이션은 이미 멀티스레드다
//   톰캣이 요청 하나를 스레드 하나에 배정한다
//   → 스프링 빈은 기본이 싱글턴이므로, 여러 요청 스레드가 같은 객체를 동시에 쓴다
//   → 그래서 빈에 요청마다 달라지는 상태를 필드로 두면 안 된다
```

```
Thread.currentThread().getName();    // http-nio-8080-exec-3 ...
```

동시성을 배우는 이유는 스레드를 직접 만들기 위해서가 아니다

프레임워크가 이미 만들어 놓은 멀티스레드 환경 위에서 '무엇을 조심해야 하는지'를 알기 위해서다.

스레드 만들기 — Thread · Runnable · Callable

- 값을 돌려받아야 한다면 Runnable 이 아니라 Callable 을 쓴다

예전 방식

```
// 1) Thread 상속 — 잘 쓰지 않는다
class Worker extends Thread {
    @Override public void run() { ... }
}
new Worker().start();

// 2) Runnable 구현 — 권장
Thread t = new Thread(() -> System.out.println("작업"));
t.start();      // run() 을 직접 부르면 새 스레드가 아니다
t.join();       // 끝날 때까지 기다린다
```

값을 돌려받아야 한다면 Callable + Future

```
Callable<Integer> task = () -> 1 + 2;

ExecutorService es = Executors.newFixedThreadPool(4);
Future<Integer> future = es.submit(task);

Integer result = future.get();    // 끝날 때까지 블로킹
es.shutdown();

// Runnable   : 값 없음 · 예외 던질 수 없음
// Callable   : 값 있음  · 예외 던질 수 있음
```

1. start() 와 run() 은 완전히 다르다

- run() 을 직접 호출하면 그냥 메서드 호출이라 현재 스레드에서 실행된다. 새 스레드에서 돌리려면 반드시 start() 를 부른다.

공유 자원과 동기화

- `count++` 처럼 단순해 보이는 연산도 원자적이지 않다

무엇이 문제인가

```
class Counter {
    private int count;
    public void increase() { count++; }    // 원자적이지 않다
}

// count++ 는 읽고 · 더하고 · 쓰는 세 단계다
// 두 스레드가 겹치면 하나가 사라진다

// 1000개 스레드가 각각 1씩 증가시켜도
// 결과가 1000 이 아니다
```

해결 수단

```
// 1) synchronized – 한 번에 하나만 들어온다
public synchronized void increase() { count++; }

// 2) 원자 클래스 – 카운터라면 이쪽이 더 빠르다
private final AtomicInteger count = new AtomicInteger();
count.incrementAndGet();

// 3) 명시적 락 – 타임아웃·인터럽트 제어가 가능
private final ReentrantLock lock = new ReentrantLock();
lock.lock(); try { ... } finally { lock.unlock(); }
```

가장 좋은 동기화는 '공유 상태를 만들지 않는 것'이다

불변 객체(record)를 쓰고, 값을 바꾸는 대신 새 값을 만들면 동기화할 것 자체가 없다.

꼭 공유해야 한다면 `ConcurrentHashMap` · `CopyOnWriteArrayList` 같은 동시성 컬렉션을 먼저 검토한다.

ExecutorService 와 스레드 풀

- 스레드를 직접 만들지 않고 작업만 던진다 — 개수 관리는 풀에 맡긴다

스레드 풀 — 스레드를 매번 만들지 않고 빌려 쓴다

```
ExecutorService es = Executors.newFixedThreadPool(10); // 스레드 10개를 미리 만들어 재사용

es.submit(() -> doWork()); // 작업만 던진다 — 스레드 관리는 풀이 한다
List<Future<Integer>> results = es.invokeAll(tasks);

es.shutdown(); // 새 작업은 안 받고, 남은 작업은 마친다
es.awaitTermination(10, TimeUnit.SECONDS);

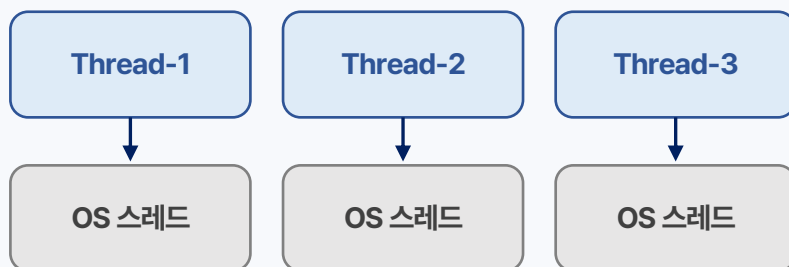
// try-with-resources 로도 쓸 수 있다 (Java 19+ • ExecutorService 가 AutoCloseable)
try (var pool = Executors.newFixedThreadPool(10)) {
    pool.submit(() -> doWork());
} // 블록을 나갈 때 shutdown + 대기
```

팩터리 메서드	특징
<code>newFixedThreadPool(n)</code>	고정 개수 — 가장 흔하다. 큐가 무한이라 작업이 밀리면 메모리를 먹는다
<code>newCachedThreadPool()</code>	필요할 때마다 만들고 60초 놓면 회수 — 짧은 작업이 몰릴 때
<code>newSingleThreadExecutor()</code>	하나씩 순서대로 — 순서 보장이 필요할 때
<code>newVirtualThreadPerTaskExecutor()</code>	작업마다 가상 스레드 하나 (Java 21) — 풀이 아니다

[도식] 플랫폼 스레드 vs 가상 스레드

- '요청 하나에 스레드 하나'라는 단순한 코드를, 스레드 풀 걱정 없이 그대로 쓰기 위한 기능

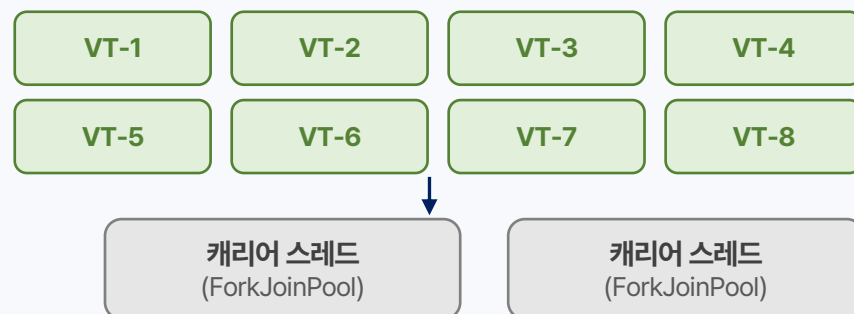
플랫폼 스레드 (기준) — 자바 스레드 1 : OS 스레드 1



OS 스레드는 비싸다

- 스택 메모리 약 1MB 를 통째로 잡는다
- 수천 개를 만들면 메모리와 컨텍스트 스위칭이 병목
- 그래서 스레드 풀로 개수를 제한해 왔다
- I/O 대기 중에도 OS 스레드를 붙잡고 있다

가상 스레드 (Java 21) — 자바 스레드 N : OS 스레드 M



가상 스레드는 싸다

- JVM 이 관리하는 객체 — 수십만 개도 만들 수 있다
- I/O 로 블로킹되면 캐리어에서 내려온다(unmount)
- 그동안 캐리어는 다른 가상 스레드를 실행한다

'요청 하나에 스레드 하나'라는 단순한 코드를, 스레드 풀 걱정 없이 그대로 쓰기 위한 기능

CPU 를 오래 쓰는 계산 작업에는 이득이 없다. I/O 대기가 많은 작업(DB · 외부 API 호출)에서 효과가 크다.

가상 스레드 (Java 21)

- 수십만 개를 만들 수 있는 경량 스레드 — Java 21 에서 정식 기능이 됐다

만들기

```
// 1) 바로 시작
Thread.startVirtualThread(() -> System.out.println("VT"));

// 2) 빌더로
Thread t = Thread.ofVirtual().name("worker-1").start(task);
t.join();

// 3) 실무에서 가장 많이 쓰는 형태
try (var es = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var id : ids) {
        es.submit(() -> callExternalApi(id)); // 1만 개 OK
    }
} // 모든 작업이 끝날 때까지 기다린 뒤 닫힌다
```

무엇이 달라지나

- 스레드 개수를 튜닝하던 일이 사라진다
→ 작업 하나에 가상 스레드 하나
- 블로킹 코드를 그대로 써도 된다
→ 리액티브로 알아엮지 않아도 처리량이 올라간다
- Spring Boot 3.2+ 에서는 설정 한 줄
spring.threads.virtual.enabled: true
→ 톰캣 요청 처리와 @Async 가 가상 스레드로
- 단, CPU 를 오래 쓰는 계산 작업에는 이득이 없다

가상 스레드는 풀에 담지 않는다

‘비싸니까 재사용한다’가 스레드 풀의 전제였는데, 가상 스레드는 싸다. 작업마다 새로 만들고 끝나면 버리는 것이 정상적인 사용법이다.

[참고] 가상 스레드 도입 시 주의점

- 설정 한 줄로 커지지만, 커기 전에 확인해야 할 것이 있다

주의점	무슨 일이 생기나	어떻게 대응하나
고정(pinning)	synchronized 블록 안에서 블로킹하면 캐리어 스레드를 붙잡고 있어 이점이 사라진다	그 구간을 ReentrantLock 으로 바꾼다 (Java 24 부터는 대부분 해소)
ThreadLocal 남용	수십만 개의 가상 스레드가 각자 값을 들고 있으면 메모리가 커진다	꼭 필요한 곳만 · 요청 컨텍스트는 명시적으로 넘긴다
커넥션 풀 병목	스레드는 늘었는데 DB 커넥션은 그대로라 그 앞에서 밀린다	실제 한계는 DB·외부 API 다 — 풀 크기와 타임아웃을 함께 본다
CPU 바운드 작업	계산 자체가 오래 걸리는 작업은 빨라지지 않는다	그런 작업은 기존 고정 크기 스레드 풀이 맞다

가상 스레드는 '동시성 문제를 없애 주는 기능'이 아니다

여전히 공유 자원에는 동기화가 필요하고, 경합 조건도 그대로 존재한다. 달라진 것은 '스레드를 아껴 쓸 필요가 없어졌다'는 점뿐이다.

CompletableFuture

- 여러 비동기 작업을 병렬로 던지고 결과를 조합한다

CompletableFuture – 여러 비동기 작업을 조합한다

```
// 비동기로 실행하고, 결과가 오면 이어서 처리한다
CompletableFuture<User> userF = CompletableFuture.supplyAsync(() -> userApi.find(id));
CompletableFuture<Order> orderF = CompletableFuture.supplyAsync(() -> orderApi.find(id));

// 두 결과를 합친다 – 두 호출이 병렬로 나간다
CompletableFuture<Result> result = userF.thenCombine(orderF, Result::of);

// 이어 붙이기
userF.thenApply(User::name)                // 값 → 값
      .thenCompose(name -> lookupAsync(name)) // 값 → 또 다른 Future
      .thenAccept(System.out::println)      // 값 → 소비
      .exceptionally(e -> { log.error("실패", e); return null; });

result.join();    // 결과를 기다린다 (Checked 예외를 던지지 않는 get)
```

가상 스레드가 나온 뒤에도 CompletableFuture 가 필요한 이유

‘여러 작업을 병렬로 던지고 결과를 합친다’는 조합(orchestration)은 여전히 CompletableFuture 의 몫이다.

다만 단순히 ‘블로킹을 피하려고’ 쓰던 코드라면, 가상 스레드로 훨씬 단순하게 표현할 수 있다.

[핵심 요약] 동시성과 가상 스레드

스레드

힙을 공유하고 스택은 각자 — 그래서 동기화가 필요하다
웹 애플리케이션은 이미 멀티스레드 환경이다

공유 상태

가장 좋은 해법은 공유 상태를 만들지 않는 것
불변 객체 · Atomic · 동시성 컬렉션 순으로 검토

가상 스레드 (Java 21)

작업마다 하나씩 만들고 버린다 — 풀링하지 않는다
I/O 대기가 많은 서버에서 처리량이 크게 오른다

주의

synchronized 블로킹 구간 · ThreadLocal 남용
진짜 한계는 대개 DB 커넥션 풀에 있다

가상 스레드의 목표는 '요청 하나에 스레드 하나'라는 가장 단순한 코드를, 성능 걱정 없이 그대로 쓰게 하는 것이다

표준 라이브러리와 빌드

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

날짜와 시간 — java.time

- Java 8 에서 새로 만든 불변 날짜/시간 API — 예전 **Date · Calendar** 는 쓰지 않는다

타입	무엇을 담나	예
LocalDate	날짜만 (연-월-일)	생일 · 마감일
LocalTime	시간만 (시:분:초)	영업 시작 시각
LocalDateTime	날짜 + 시간 (시간대 없음)	게시글 작성 시각(단일 지역 서비스)
ZonedDateTime	날짜 + 시간 + 시간대	여러 지역 사용자를 다루는 예약 시스템
Instant	UTC 기준 시점(타임스탬프)	로그 · 이벤트 발생 시각 · DB 저장
Duration / Period	시간 간격 / 날짜 간격	소요 시간 / 나이 계산

java.time 은 모두 불변이다

```

LocalDate today = LocalDate.now();
LocalDate due   = today.plusDays(7);           // 불변 - 새 객체가 돌아온다
boolean over    = due.isBefore(LocalDate.now());

String s = today.format(DateTimeFormatter.ofPattern("yyyy-MM-dd"));
LocalDate parsed = LocalDate.parse("2026-07-30");
long days = ChronoUnit.DAYS.between(today, due);

```

Date · Calendar · SimpleDateFormat 은 쓰지 않는다

가변이라 스레드 안전하지 않고 월이 0부터 시작하는 등 함정이 많다.

파일 입출력 — Path 와 Files

- Path 로 경로를 만들고 Files 의 정적 메서드로 읽고 쓴다

파일 읽기 · 쓰기 (java.nio.file)

```
Path path = Path.of("data", "users.csv");

String text = Files.readString(path);          // 전체를 문자열로
List<String> lines = Files.readAllLines(path);

Files.writeString(path, "내용");              // 덮어쓰기
Files.writeString(path, "추가", StandardOpenOption.APPEND);

Files.exists(path);   Files.createDirectories(dir);
Files.delete(path);   Files.size(path);
```

큰 파일은 스트림으로 한 줄씩

```
try (var lines = Files.lines(path)) {          // 반드시 닫는다
    long n = lines.filter(l -> l.contains("ERROR")).count();
}

// 전체를 메모리에 올리지 않으므로 수 GB 파일도 처리 가능

// 디렉터리 순회
try (var paths = Files.walk(root)) {
    paths.filter(f -> f.toString().endsWith(".java"))
           .forEach(System.out::println);
}
```

File 대신 Path, FileInputStream 대신 Files 를 먼저 떠올린다

Files 의 정적 메서드만으로 대부분의 파일 작업이 한 줄로 끝난다. 스트림을 돌려주는 메서드(lines · walk)는 try-with-resources 로 닫아야 한다.

어노테이션과 리플렉션

- 코드에 정보를 붙이고, 실행 중에 그 정보를 읽어 동작을 바꾼다 — 스프링의 토대

어노테이션 – 코드에 붙이는 메타데이터

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)    // 런타임에도 읽기
public @interface Timed {
    String value() default "";
}

@Timed("주문 저장")
public void save(Order order) { ... }

// RetentionPolicy 가 SOURCE 나 CLASS 면 실행 중에는 볼 수 없다
```

리플렉션 – 실행 중에 클래스 정보를 읽는다

```
Method m = obj.getClass().getMethod("save", Order.class);

if (m.isAnnotationPresent(Timed.class)) {
    Timed t = m.getAnnotation(Timed.class);
    long start = System.nanoTime();
    m.invoke(obj, order);
    log.info("{} {}ns", t.value(), System.nanoTime()-start);
}
```

스프링의 정체는 여기 있다

@Component 를 붙인 클래스를 찾아 빈으로 등록하고, @Transactional 이 붙은 메서드를 프록시로 감싸는 일 전부가 '어노테이션 + 리플렉션'이다.

직접 리플렉션을 쓸 일은 드물지만, 프레임워크가 무엇을 하고 있는지 이해하려면 이 구조를 알아야 한다.

Gradle 프로젝트 구조와 빌드

- 소스 위치와 빌드 설정을 표준에 맞추면 도구가 알아서 찾아 준다

표준 프로젝트 구조

```

my-app/
├── build.gradle      빌드 설정 · 의존성
├── settings.gradle   프로젝트 이름
├── gradlew           래퍼 (버전 고정)
├── src/
│   ├── main/java/... 애플리케이션 코드
│   ├── main/resources/ 설정 파일 · 정적 자원
│   ├── test/java/... 테스트 코드
│   └── test/resources/

```

build.gradle

```

plugins { id 'java' }

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories { mavenCentral() }

dependencies {
    testImplementation platform('org.junit:junit-bom:5.x')
    testImplementation 'org.junit.jupiter:junit-jupiter'
}

tasks.named('test') { useJUnitPlatform() }

```

toolchain 을 지정하면 팀원의 로컬 JDK 버전이 달라도 같은 결과가 나온다

Gradle 이 21 을 찾아 쓰거나 없으면 내려받는다. `./gradlew build` · `./gradlew test` · `./gradlew bootRun` 이 가장 자주 쓰는 명령이다.

JAR · 모듈 시스템 · jlink

- 클래스를 묶어 배포하고, 필요하면 런타임까지 잘라서 만든다

JAR — 클래스 파일을 묶은 압축 파일

```
$ jar --create --file app.jar -C build/classes .
$ jar --list --file app.jar
$ java -jar app.jar          # MANIFEST 에 Main-Class 필요
```

```
app.jar
├── META-INF/MANIFEST.MF      Main-Class · Class-Path
├── com/sk/skala/App.class
└── application.yml
```

// Spring Boot 의 실행 가능 JAR 도 구조만 조금 다른 JAR 다

모듈 시스템 (Java 9+)

```
// src/main/java/module-info.java
module com.sk.skala.app {
    requires java.sql;           // 내가 쓰는 모듈
    exports com.sk.skala.api;    // 내가 공개하는 패키지
}
```

// 공개하지 않은 패키지는 밖에서 접근할 수 없다

```
// jlink - 필요한 모듈만 골라 작은 런타임을 만든다
$ jlink --add-modules java.base --output myruntime
```

일반적인 스프링 부트 프로젝트는 모듈 시스템을 쓰지 않는다

라이브러리 호환 문제가 있어 대부분 클래스패스 방식을 그대로 쓴다. 다만 JDK 자체가 모듈로 나뉘어 있다는 사실은 알아 둘 필요가 있다
— 컨테이너 이미지 크기를 줄일 때 jlink 로 만든 커스텀 런타임이 쓰인다.

JUnit 5 단위 테스트

- 테스트는 '이 코드가 무엇을 보장하는가'를 실행 가능한 형태로 적어 둔 문서다

JUnit 5 - 단위 테스트

```
class OrderServiceTest {  
  
    @Test  
    @DisplayName("수량이 0 이하면 주문을 만들 수 없다")  
    void createOrder_invalidQty() {  
        assertThrows(IllegalArgumentException.class,  
            () -> new Order("A1", 0, 1000));  
    }  
  
    @Test  
    void total_계산() {  
        var order = new Order("A1", 3, 1000);  
        assertEquals(3000, order.total());  
    }  
  
    @ParameterizedTest  
    @ValueSource(ints = {-1, 0})  
    void 잘못된_수량(int qty) {  
        assertThrows(IllegalArgumentException.class, () -> new Order("A1", qty, 1000));  
    }  
}
```

자주 쓰는 단언 : assertEquals · assertTrue · assertThrows · assertAll · assertNull

테스트 이름은 '어떤 상황에서 어떻게 동작해야 하는가'를 문장으로 적는다. @DisplayName 을 쓰면 한글로 쓸 수 있다.

[핵심 요약] 표준 라이브러리와 빌드

날짜와 시간

java.time 은 전부 불변 — Date · Calendar 는 쓰지 않는다
시간대가 의미 있으면 ZonedDateTime · 저장은 Instant

파일 입출력

Path + Files 정적 메서드로 대부분 한 줄
Files.lines · Files.walk 는 반드시 닫는다

어노테이션과 리플렉션

스프링의 자동 등록·프록시가 서 있는 토대
직접 쓸 일은 드물지만 원리는 알아 둔다

빌드와 테스트

Gradle toolchain 으로 JDK 21 을 고정한다
JUnit 5 로 '상황 → 기대 결과'를 문장처럼 적는다

표준 라이브러리를 알수록 직접 짜는 코드가 줄고, 직접 짜는 코드가 줄수록 버그도 줄어든다

정말 정말~~~ 수고하셨습니다.^^
(여러분이 SKALA 의 주인공입니다. 끝까지 화이팅!!!👍)

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.